Trilhas, competências e questionários

ITT — avaliação de proficiência pós-trilha

Relatório de referência para a coordenação: o que cada trilha da ITT se propõe a ensinar, quais competências o questionário pós-trilha cobra, o que cada curso da trilha contribui, e o prompt pronto para gerar o questionário na plataforma parceira.

Trilhas cobertas	9 (todas as da planilha atual da ITT)
Cursos	39
Carga total	329h
Itens por questionário	20, sendo 2 dissertativos
Nível mínimo esperado	adequado (70–84%)
Gerado em	2026-10-21
Gerado por	`scripts/gerar_relatorio.py`

> **Como ler este relatório.** O questionário é **um por trilha**, não um por curso: a avaliação acontece depois de a trilha inteira ser concluída e mede competências que atravessam vários cursos. Por isso cada trilha tem *um* prompt, e a contribuição de cada curso aparece nos seus objetivos de aprendizagem, que apontam para as competências cobradas.

Sumário

Panorama das trilhas **1**
Como aplicar . . . 1
Por que três botões e três tipos . . . 1

PR01 — A partir do zero: iniciante em programação **2**
Competências avaliadas . . . 2
Cursos da trilha . . . 4
PR01.01 Começando em Programação: carreira e primeiros passos . . . 4
PR01.02 Lógica de programação: mergulhe em programação com JavaScript . . . 5
PR01.03 Lógica de programação: explore funções e listas . . . 5
PR01.04 Git e GitHub: compartilhando e colaborando em projetos . . . 6
PR01.05 Lógica de programação: praticando com desafios . . . 7
Rubrica de proficiência . . . 7
Prompt do questionário . . . 8

PR02 — Aprenda a programar em Python com Orientação a Objetos **11**
Competências avaliadas . . . 11
Cursos da trilha . . . 13
PR02.01 Python: crie a sua primeira aplicação . . . 14
PR02.02 Python: aplicando a Orientação a Objetos . . . 14
PR02.03 Python: avance na Orientação a Objetos e consuma API . . . 15
Rubrica de proficiência . . . 16
Prompt do questionário . . . 16

PR03 — Django: crie aplicações em Python **20**
Competências avaliadas . . . 20
Cursos da trilha . . . 22
PR03.01 Django: templates e boas práticas . . . 23
PR03.02 Django: persistência de dados e Admin . . . 23
PR03.03 Django: autenticação de formulários e alerta . . . 24
PR03.04 Django: autenticação com OAuth2.0 . . . 24
PR03.05 Django: CRUD e persistência no S3 . . . 25
Rubrica de proficiência . . . 25
Prompt do questionário . . . 26

PR06 — Desenvolva jogos com Unity **30**
Competências avaliadas . . . 30
Cursos da trilha . . . 32
PR06.01 Unity: criando um jogo metroidvania 2D . . . 33

PR06.02 Unity: criando menus, coletáveis e batalha final em um jogo 2D 33
Rubrica de proficiência 34
Prompt do questionário 35

PR12 — Boas práticas em Python **38**
Competências avaliadas 38
Cursos da trilha 40
PR12.01 Python: aplicando boas práticas com PEP 8 40
PR12.02 Python: construindo sistemas eficientes com SOLID e Design Patterns 41
PR12.03 Python e TDD: explorando testes unitários 42
Rubrica de proficiência 42
Prompt do questionário 43

FE01 — A partir do zero: HTML e CSS para projetos web **46**
Competências avaliadas 46
Cursos da trilha 48
FE01.01 Internet: entendendo os fundamentos da web 48
FE01.02 HTML e CSS: ambiente, estrutura e estilo 49
FE01.03 HTML e CSS: formulários, SEO e acessibilidade 50
FE01.04 HTML e CSS: responsividade em layouts 50
Rubrica de proficiência 51
Prompt do questionário 52

FE03 — Desenvolvimento Front-end: criar aplicações web com HTML, CSS e JavaScript **55**
Competências avaliadas 55
Cursos da trilha 57
FE03.01 JavaScript: construindo páginas dinâmicas 58
FE03.02 JavaScript: entendendo promises e async/await 58
FE03.03 JavaScript: métodos de array 59
FE03.04 JavaScript: manipulando elementos no DOM 59
FE03.05 JavaScript: explorando a manipulação de elementos e da localStorage 60
FE03.06 JavaScript: implementando CRUD com requisições HTTP 60
FE03.07 JavaScript: evoluindo a sua aplicação com ES6+ 61
FE03.08 Node.js e terminal: dominando o ambiente de desenvolvimento front-end . . . 62
Rubrica de proficiência 62
Prompt do questionário 63

UX02 — Game Design **66**
Competências avaliadas 66
Cursos da trilha 68

UX02.01 Game Design: definindo os princípios de um jogo digital 69
UX02.02 Game Design: viabilizando o jogo como produto 69
UX02.03 Game design: refinando aspectos visuais e artísticos do jogo 70
UX02.04 Game Design: entenda estratégias para viabilizar e monetizar o seu jogo . . . 71
Rubrica de proficiência 71
Prompt do questionário 72

UX03 — Blender **75**
Competências avaliadas 75
Cursos da trilha 77
UX03.01 Blender: criando um modelo 3D 78
UX03.02 Blender: entendendo o Box Modeling 78
UX03.03 Blender: texturizando modelos com Hand Paint 79
UX03.04 Blender: modelando com a técnica Poly by Poly 79
UX03.05 Blender: riggando e animando seu personagem 80
Rubrica de proficiência 81
Prompt do questionário 81

Apêndice — fontes e reprodutibilidade **85**

Panorama das trilhas

ID	Escola	Trilha	Carga	Cursos	Itens
PR01	Programação	A partir do zero: iniciante em programação	29h	5	20
PR02	Programação	Aprenda a programar em Python com Orientação a Objetos	22h	3	20
PR03	Programação	Django: crie aplicações em Python	44h	5	20
PR06	Programação	Desenvolva jogos com Unity	24h	2	20
PR12	Programação	Boas práticas em Python	24h	3	20
FE01	Front-end	A partir do zero: HTML e CSS para projetos web	54h	4	20
FE03	Front-end	Desenvolvimento Front-end: criar aplicações web com HTML, CSS e JavaScript	60h	8	20
UX02	UX & Design	Game Design	32h	4	20
UX03	UX & Design	Blender	40h	5	20
Total			**329h**	**39**	**180**

Como aplicar

1. Abra a seção **Prompt do questionário** da trilha e cole o **texto para colar** no campo de prompt da plataforma, uma vez só.
2. Percorra o **roteiro de montagem** linha por linha. Em cada linha, selecione na plataforma o botão de tipo indicado — múltipla escolha, múltipla resposta ou dissertativa — e peça a quantidade daquele tema.
3. Confira os totais ao fim do roteiro antes de fechar o questionário.
4. Aplique depois de a trilha estar concluída e reporte a faixa da **rubrica de proficiência** da trilha.

Por que três botões e três tipos

A matriz de cada trilha classifica os itens em múltipla escolha, análise de código e dissertativa. A plataforma parceira, porém, oferece outros três botões: múltipla escolha, múltipla resposta e dissertativa. O roteiro faz a tradução: os itens de análise de código viram múltipla escolha com um trecho de código no enunciado (coluna **Com código**), e parte dos itens objetivos vira múltipla resposta onde o tema pede identificar um conjunto.

Nas duas trilhas sem programação — UX02 Game Design e UX03 Blender — o tipo análise de código não se aplica, e a coluna passa a ser **Com estímulo**: um trecho curto de Game Design Document, uma cena ou uma situação de modelagem descrita em texto. Nenhum prompt pede que o estudante veja uma imagem.

PR01 — A partir do zero: iniciante em programação

Escola (ITT)	Programação
Nome na planilha ITT	A partir do zero: iniciante em programação
Nome atual na Alura	A partir do zero: iniciante em programação
Página da formação	`https://www.alura.com.br/formacao-programacao`
Carga horária	29h em 5 cursos
Pré-requisitos	nenhum
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, sem nenhuma experiência prévia em programação
Projeto fio condutor	Pequenos projetos em JavaScript rodando no navegador (jogo de adivinhação, AluGames, carrinho de compras, amigo secreto), publicados no GitHub
Matriz	`cursos/PR01-iniciante-programacao.json`
Prompt	`prompts/PR01.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`PR01.C1`	Situar a programação como área: o que é, onde se aplica e quais são as frentes de atuação	0.08	2	entender
`PR01.C2`	Usar variáveis, tipos e entrada/saída em JavaScript	0.17	3	aplicar
`PR01.C3`	Tomar decisões no código com condicionais e operadores	0.20	4	aplicar
`PR01.C4`	Repetir ações com laços	0.17	3	aplicar
`PR01.C5`	Organizar o código em funções com parâmetros e retorno	0.15	3	aplicar
`PR01.C6`	Guardar e percorrer conjuntos de dados em listas	0.13	3	aplicar
`PR01.C7`	Versionar e publicar um projeto com Git e GitHub	0.10	2	aplicar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

PR01.C1 — Situar a programação como área: o que é, onde se aplica e quais são as frentes de atuação

Reconhecer o que um programa faz, distinguir as grandes áreas (front-end, back-end, mobile, dados) e identificar o papel das ferramentas básicas do desenvolvedor.

Evidências observáveis

- Dado um exemplo de produto digital, aponta qual frente da programação o constrói
- Explica em uma frase o que é um algoritmo
- Distingue linguagem de programação de ferramenta (editor, navegador, repositório)

Tipos de item: `multipla_escolha` **Cursos de origem:** `PR01.01`

PR01.C2 — Usar variáveis, tipos e entrada/saída em JavaScript

Declarar variáveis com let e const, atribuir e reatribuir valores, distinguir número de texto e exibir informação na tela de forma interativa.

Evidências observáveis

- Prevê o valor final de uma variável após uma sequência de atribuições
- Escolhe entre let e const conforme o valor mudar ou não
- Monta uma mensagem concatenando texto com o conteúdo de uma variável

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR01.02`

PR01.C3 — Tomar decisões no código com condicionais e operadores

Escrever if/else e if/else if encadeados, combinar condições com operadores lógicos e comparar valores corretamente.

Evidências observáveis

- Traduz uma regra em linguagem natural ('se a nota for 7 ou mais, aprovado') para um if correto
- Prevê qual ramo de um if/else if executa para um dado valor
- Combina duas condições com && ou || conforme o enunciado

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR01.02`, `PR01.05`

PR01.C4 — Repetir ações com laços

Escolher e escrever for ou while conforme o problema, definir corretamente a condição de parada e acumular resultados dentro do laço.

Evidências observáveis

- Conta quantas vezes um laço executa para um dado intervalo
- Justifica o uso de while quando o número de repetições não é conhecido de antemão
- Rastreia o valor de um acumulador ao longo das iterações

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR01.02`, `PR01.05`

PR01.C5 — Organizar o código em funções com parâmetros e retorno

Criar funções que recebem dados, devolvem resultado e evitam repetição; chamar funções passando argumentos e usar o valor retornado.

Evidências observáveis

- Identifica em um código qual trecho repetido deveria virar função
- Distingue o que a função devolve (return) do que ela mostra na tela
- Prevê a saída de uma chamada de função com argumentos diferentes

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR01.03`, `PR01.05`

PR01.C6 — Guardar e percorrer conjuntos de dados em listas

Criar arrays, acessar elementos por índice, adicionar e remover itens, percorrer a lista e tomar decisões sobre seus elementos.

Evidências observáveis

- Acessa o elemento correto dado um índice
- Prevê o conteúdo da lista após push/splice e o valor de length
- Combina laço e condicional para filtrar elementos de uma lista

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR01.03`, `PR01.05`

PR01.C7 — Versionar e publicar um projeto com Git e GitHub

Criar um repositório, registrar alterações em commits com mensagem, sincronizar com o GitHub e entender o fluxo de colaboração.

Evidências observáveis

- Coloca em ordem os passos de salvar uma alteração no GitHub (add, commit, push)
- Explica para que serve um commit em uma frase
- Reconhece o que é um conflito e quando ele aparece

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `PR01.04`, `PR01.03`

Cursos da trilha

PR01.01 — Começando em Programação: carreira e primeiros passos

1h · 7 atividades · atualizado em 2024-11-06 | Etapa: *Primeiros passos em Programação*
`https://www.alura.com.br/curso-online-comecando-programacao-carreira-primeiros-passos`

Módulos da ementa. Começando em Programação; O que é Programação?; Áreas de atuação; Mercado de trabalho e desenvolvimento contínuo; Desenvolvimento contínuo.

Conceitos-chave. `programação`, `algoritmo`, `front-end`, `back-end`, `mobile`, `carreira em tecnologia`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR01.01.O1	Explicar o que é programação e o que um programa faz	entender	PR01.C1
PR01.01.O2	Compreender os fundamentos e as áreas da programação (front-end, back-end, mobile, entre outras)	entender	PR01.C1
PR01.01.O3	Identificar os próximos passos nos estudos e o que o mercado espera de quem começa	entender	PR01.C1

Competências alimentadas por este curso: PR01.C1 Situar a programação como área: o que é, onde se aplica e quais são as frentes de atuação.

Fora de escopo (o questionário não cobra). salários específicos; comparação entre linguagens; processos seletivos.

PR01.02 — Lógica de programação: mergulhe em programação com JavaScript

6h · 51 atividades · atualizado em 2026-07-17 | Etapa: *Entendendo a lógica de programação*
https://www.alura.com.br/curso-online-logica-programacao-mergulhe-programacao-javascript

Módulos da ementa. Iniciando com JavaScript; Carregando o projeto base; Exibindo mensagens na tela; Comparando variáveis; Condicionais e concatenação; Loops e tentativas; Boas práticas de programação; Desafio.

Conceitos-chave. variável, let, const, alert, prompt, console.log, concatenação, if/else, operadores de comparação, while, for, boas práticas.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR01.02.O1	Exibir mensagens na tela de forma interativa e ler dados da pessoa usuária	aplicar	PR01.C2
PR01.02.O2	Utilizar variáveis para guardar e atualizar informação durante a execução	aplicar	PR01.C2
PR01.02.O3	Comparar valores e decidir o caminho do programa com condicionais	aplicar	PR01.C3
PR01.02.O4	Repetir tentativas com laços até que uma condição de parada seja satisfeita	aplicar	PR01.C4
PR01.02.O5	Seguir boas práticas de nomenclatura e legibilidade ao escrever código	entender	PR01.C2
PR01.02.O6	Desenvolver uma aplicação simples do início ao fim, inspirada em um caso real	aplicar	PR01.C3

Competências alimentadas por este curso: PR01.C2 Usar variáveis, tipos e entrada/saída em JavaScript; PR01.C3 Tomar decisões no código com condicionais e operadores; PR01.C4 Repetir ações com laços.

Fora de escopo (o questionário não cobra). orientação a objetos; async/await; frameworks; APIs externas.

PR01.03 — Lógica de programação: explore funções e listas

6h · 44 atividades · atualizado em 2025-08-18 | Etapa: *Entendendo a lógica de programação*
https://www.alura.com.br/curso-online-logica-programacao-funcoes-listas

Módulos da ementa. Interagindo com HTML; Manipulando textos; Criando uma função; Funções; Reiniciando o jogo; Listas; Publicando o projeto.

Conceitos-chave. `função`, `parâmetro`, `argumento`, `return`, `document.querySelector`, `manipulação de texto`, `array`, `índice`, `push`, `length`, `publicação (GitHub/Vercel)`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR01.03.O1	Criar funções com parâmetros e retorno para evitar repetição de código	aplicar	PR01.C5
PR01.03.O2	Manipular elementos da tela e textos de forma dinâmica com JavaScript	aplicar	PR01.C5
PR01.03.O3	Utilizar listas para armazenar vários valores e percorrê-las	aplicar	PR01.C6
PR01.03.O4	Combinar lista e condicional para controlar o estado de um jogo (ex.: não repetir sorteios)	analisar	PR01.C6
PR01.03.O5	Publicar o projeto na web usando GitHub e Vercel	aplicar	PR01.C7

Competências alimentadas por este curso: PR01.C5 Organizar o código em funções com parâmetros e retorno; PR01.C6 Guardar e percorrer conjuntos de dados em listas; PR01.C7 Versionar e publicar um projeto com Git e GitHub.

Fora de escopo (o questionário não cobra). map/filter/reduce; objetos literais complexos; CSS avançado; frameworks front-end.

PR01.04 — Git e GitHub: compartilhando e colaborando em projetos

8h · 66 atividades · atualizado em 2026-03-17 | Etapa: *Ferramentas essenciais para programar e compartilhar projetos*
`https://www.alura.com.br/curso-online-git-github-compartilhando-colaborando-projetos`

Módulos da ementa. Compartilhando projetos; Conheça o GitHub; Criando um repositório; Instalando o Git; Sincronizando repositórios; Colaborando em projetos; Utilizando Git na IDE; Voltando no tempo; Mais recursos.

Conceitos-chave. `repositório`, `commit`, `add`, `push`, `pull`, `clone`, `branch`, `merge`, `conflito`, `pull request`, `histórico`, `portfólio`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR01.04.O1	Criar um repositório e versionar um projeto de software com Git	aplicar	PR01.C7
PR01.04.O2	Compartilhar o código no GitHub e montar um portfólio de projetos	aplicar	PR01.C7
PR01.04.O3	Colaborar em projetos sincronizando repositórios local e remoto	aplicar	PR01.C7
PR01.04.O4	Reconhecer um conflito de código e entender como resolvê-lo	analisar	PR01.C7
PR01.04.O5	Analisar e modificar o histórico de commits de um repositório	analisar	PR01.C7

Competências alimentadas por este curso: PR01.C7 Versionar e publicar um projeto com Git e GitHub.

Fora de escopo (o questionário não cobra). rebase interativo; GitHub Actions; submodules; estratégias de branching corporativas.

PR01.05 — Lógica de programação: praticando com desafios

8h · 70 atividades · atualizado em 2025-03-18 | Etapa: *Praticando Lógica de Programação com desafios* https://www.alura.com.br/curso-online-logica-programacao-praticando-desafios

Módulos da ementa. Projeto AluGames; Obtendo o jogo clicado e alterando seu status; Projeto Carrinho de Compras; Projeto Ingresso Online; Projeto Amigo Secreto; Desafios complementares.

Conceitos-chave. `decomposição de problema`, `estado`, `evento de clique`, `cálculo com dados da tela`, `validação de entrada`, `sorteio sem repetição`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR01.05.O1	Decompor um enunciado de problema em passos implementáveis	analisar	PR01.C3
PR01.05.O2	Implementar soluções completas combinando variáveis, condicionais, laços, funções e listas	aplicar	PR01.C5
PR01.05.O3	Controlar o estado de uma aplicação a partir da interação da pessoa usuária	aplicar	PR01.C6
PR01.05.O4	Identificar e corrigir erros de lógica em um código que não produz o resultado esperado	analisar	PR01.C4

Competências alimentadas por este curso: PR01.C3 Tomar decisões no código com condicionais e operadores; PR01.C4 Repetir ações com laços; PR01.C5 Organizar o código em funções com parâmetros e retorno; PR01.C6 Guardar e percorrer conjuntos de dados em listas.

Fora de escopo (o questionário não cobra). persistência de dados; backend; testes automatizados; acessibilidade avançada.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece termos isolados, mas não consegue prever o que um código simples faz nem traduzir uma regra em condicional.
basico	50–69	Lê código com variáveis e if/else e acerta saídas diretas; erra em laços com acumulador, funções com retorno e índices de lista.
adequado	70–84	Traduz um enunciado em código com variáveis, condicionais, laços e listas; cria funções com parâmetros e sabe versionar o projeto no GitHub.
avancado	85–100	Decompõe problemas novos, depura erros de lógica em código alheio e justifica escolhas (for vs while, quando extrair função).

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 40 minutos estimados. Por tipo na matriz: multipla_escolha 12 · analise_codigo 6 · dissertativa 2. Por nível cognitivo: lembrar/entender 6 · aplicar 10 · analisar/avaliar 4.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 Programação como área	2	Múltipla escolha	0	frentes de atuação, o que é um algoritmo
C2 Variáveis, tipos, entrada/saída	3	Múltipla escolha	1	let vs const, concatenação, prever valor final
C3 Condicionais e operadores	3	Múltipla escolha	2	traduzir regra em if, prever qual ramo executa
C3 Condicionais	1	Dissertativa	0	justificar por que um encadeamento de ifs está errado
C4 Laços	3	Múltipla escolha	1	contagem de iterações, acumulador, for vs while
C5 Funções	2	Múltipla escolha	1	return vs mostrar na tela, saída de uma chamada
C5 Funções	1	Dissertativa	0	quando extrair um trecho repetido para uma função
C6 Listas	2	Múltipla escolha	1	índices, length, filtrar com laço e condicional
C6 Listas	1	Múltipla resposta	0	quais afirmações sobre índices e length estão corretas
C7 Git e GitHub	1	Múltipla escolha	0	ordem de add, commit e push
C7 Git e GitHub	1	Múltipla resposta	0	quais ações levam a alteração para o GitHub

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 6 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/PR01-iniciante-programacao.json‘ (‘multipla_escolha‘ 12 + ‘analise_codigo‘ 6) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 6 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - INICIANTE EM PROGRAMAÇÃO (JAVASCRIPT)
```

```
Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. Esta é a primeira trilha de
programação do estudante - não pressuponha nenhuma outra linguagem, nem orientação a
objetos, nem HTML/CSS, nem terminal além do básico de Git. Todo código em JavaScript
rodando no navegador.

ASSUNTOS

C1 - A programação como área
  Cobrar: o que um programa faz; o que é um algoritmo em uma frase; as frentes de atuação
  (front-end, back-end, mobile, dados); diferença entre linguagem de programação e
  ferramenta (editor, navegador, repositório).
  Erros típicos, para virar alternativa errada: achar que front-end e back-end são
  linguagens e não camadas; achar que programa é só aplicativo de celular; supor que o
  navegador compila o código para outra linguagem.

C2 - Variáveis, tipos e entrada/saída
  Cobrar: prever o valor final de uma variável depois de uma sequência de atribuições;
  escolher entre let e const conforme o valor mude ou não; montar uma mensagem
  concatenando texto com o conteúdo de uma variável; ler dados e exibir resultado.
  Erros típicos: somar número com texto e obter concatenação em vez de soma; reatribuir
  uma const; usar aspas em volta do nome da variável, confundindo nome com valor; usar =
  onde queria ==.

C3 - Condicionais e operadores
  Cobrar: traduzir uma regra em linguagem natural ("se a nota for 7 ou mais, aprovado")
  para um if correto; prever qual ramo de um if / else if / else executa para um dado
  valor; combinar duas condições com && ou ||.
  Erros típicos: usar = em vez de == ou === na comparação; trocar && por || e inverter a
  lógica; encadear ifs independentes onde o caso pedia else if; ordem errada dos testes
  deixando um ramo inalcançável.

C4 - Laços
  Cobrar: contar quantas vezes um laço executa em um dado intervalo; rastrear o valor de
  um acumulador ao longo das iterações; justificar while quando o número de repetições
  não é conhecido de antemão.
  Erros típicos: laço infinito por não atualizar a variável de controle; erro de 1 a mais
  ou a menos nos limites do for; declarar o acumulador dentro do laço e zerá-lo a cada
  volta; confundir a condição de continuar com a de parar.

C5 - Funções com parâmetros e retorno
  Cobrar: identificar qual trecho repetido deveria virar função; distinguir o que a função
  devolve com return do que ela mostra na tela; prever a saída de uma chamada com
  argumentos diferentes.
  Erros típicos: definir a função e nunca chamá-la; usar o resultado de uma função que só
  imprime e não retorna; trocar a ordem dos argumentos na chamada; usar fora da função uma
  variável declarada dentro dela.

C6 - Listas
  Cobrar: acessar o elemento certo a partir de um índice; prever o conteúdo da lista
  depois de push ou splice e o valor de length; combinar laço e condicional para filtrar
  elementos.
  Erros típicos: achar que o primeiro índice é 1; acessar índice igual a length e obter
  undefined; confundir o índice com o valor armazenado; remover itens enquanto percorre a
  lista.

C7 - Git e GitHub
  Cobrar: ordenar os passos para salvar uma alteração no GitHub (add, commit, push);
  explicar para que serve um commit em uma frase; reconhecer o que é um conflito e quando
  ele aparece.
```

```
  Erros típicos: achar que salvar o arquivo no editor já versiona a mudança; confundir
  commit (local) com push (envio ao GitHub); supor que Git e GitHub são a mesma coisa;
  achar que clonar e baixar o ZIP são equivalentes.

FORA DE ESCOPO - não cobre nada disso: orientação a objetos, async/await, frameworks,
APIs externas, map/filter/reduce, arrow functions, template literals complexos, objetos
literais complexos, CSS, persistência de dados, back-end, testes automatizados, rebase
interativo, GitHub Actions e estratégias de branching corporativas.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: jogos, música, escola, time, rede social,
  lanchonete.
- Quando o item trouxer código, use JavaScript com no máximo 12 linhas, sem bibliotecas:
  só let/const, if/else, for, while, funções declaradas com function, arrays e
  console.log/alert.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie raciocínio de programação, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 6 itens de compreensão, 10 de aplicação e 4
  de análise ou avaliação.
```

PR02 — Aprenda a programar em Python com Orientação a Objetos

Escola (ITT)	Programação
Nome na planilha ITT	Aprenda a programar em Python com Orientação a Objetos
Nome atual na Alura	Aprenda a programar em Python com Orientação a Objetos
Página da formação	`https://www.alura.com.br/formacao-linguagem-python`
Carga horária	22h em 3 cursos
Pré-requisitos	PR01
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, já alfabetizados em lógica de programação (PR01), iniciando em Python
Projeto fio condutor	Sistema de cadastro e avaliação de restaurantes/cardápio construído ao longo dos três cursos
Matriz	`cursos/PR02-python-orientacao-objetos.json`
Prompt	`prompts/PR02.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`PR02.C1`	Escrever e executar um programa Python usando variáveis, strings e entrada/saída	0.12	2	aplicar
`PR02.C2`	Controlar o fluxo do programa com condicionais, laços e tratamento de exceções	0.15	3	aplicar
`PR02.C3`	Armazenar e percorrer dados em listas e dicionários	0.15	3	aplicar
`PR02.C4`	Organizar o código em funções e módulos	0.10	2	aplicar
`PR02.C5`	Modelar um domínio com classes, atributos de instância e construtor	0.18	4	aplicar
`PR02.C6`	Aplicar encapsulamento com property e métodos de classe e estáticos	0.10	2	aplicar
`PR02.C7`	Reutilizar código com herança, polimorfismo e classes abstratas	0.12	2	analisar
`PR02.C8`	Trabalhar com ambiente virtual, arquivos JSON e consumo de API	0.08	2	aplicar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

PR02.C1 — Escrever e executar um programa Python usando variáveis, strings e entrada/saída

Criar um script Python, declarar variáveis com nomes significativos, formatar saída com f-strings e ler dados do usuário com input(), respeitando a indentação da linguagem.

Evidências observáveis

- Dado um trecho com variáveis e print(), prevê corretamente a saída
- Monta uma f-string que interpola o valor de uma variável dentro de um texto
- Reconhece que input() devolve string e precisa de conversão para cálculo numérico

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR02.01`

PR02.C2 — Controlar o fluxo do programa com condicionais, laços e tratamento de exceções

Decidir entre caminhos com if/elif/else, repetir ações com for e while, e proteger o programa de entradas inválidas com try/except.

Evidências observáveis

- Dado um enunciado, escolhe entre for e while justificando pela condição de parada
- Prevê quantas vezes um laço executa
- Identifica qual bloco except captura um erro específico (ValueError, KeyError)

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR02.01`

PR02.C3 — Armazenar e percorrer dados em listas e dicionários

Escolher entre lista e dicionário conforme o dado, acessar por índice ou por chave, adicionar e remover elementos, e percorrer a coleção com for.

Evidências observáveis

- Justifica o uso de dicionário quando o dado tem rótulo (nome, categoria) e de lista quando a ordem importa
- Acessa corretamente um valor aninhado como lista de dicionários
- Prevê o resultado de append(), len() e de um for sobre a coleção

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR02.01`

PR02.C4 — Organizar o código em funções e módulos

Extrair um trecho repetido para uma função com parâmetros e retorno, e separar o programa em arquivos importáveis.

Evidências observáveis

- Distingue parâmetro de argumento e return de print
- Identifica qual trecho repetido deveria virar função
- Escreve o import correto para usar uma classe ou função de outro arquivo

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR02.01`, `PR02.02`

PR02.C5 — Modelar um domínio com classes, atributos de instância e construtor

Dado um enunciado do mundo real, identificar quais entidades viram classes, quais informações viram atributos e quais ações viram métodos; instanciar objetos pelo construtor ___init___.

Evidências observáveis

- Dado 'um jogo tem nome, nota e gênero', escreve a classe com ___init___ e os três atributos
- Explica o papel do self no construtor e nos métodos
- Distingue a classe (o molde) do objeto (a instância)

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR02.02`

PR02.C6 — Aplicar encapsulamento com property e métodos de classe e estáticos

Proteger atributos internos, expor leitura e escrita controladas com @property e @<attr>.setter, e reconhecer quando um método pertence à classe e não à instância.

Evidências observáveis

- Reconhece que @property permite rodar lógica (validação, formatação) ao ler um atributo
- Identifica a convenção do underscore (_atributo) como sinal de uso interno
- Sabe que o setter é quem valida o valor antes de atribuir

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR02.02`

PR02.C7 — Reutilizar código com herança, polimorfismo e classes abstratas

Criar uma subclasse que estende a superclasse, sobrescrever métodos para comportamentos diferentes sob a mesma interface, e reconhecer o papel de um método abstrato.

Evidências observáveis

- Dado um conjunto de classes, aponta qual deveria ser a superclasse e por quê
- Prevê qual implementação de método será executada em uma chamada polimórfica
- Explica por que uma classe abstrata não pode ser instanciada

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR02.03`

PR02.C8 — Trabalhar com ambiente virtual, arquivos JSON e consumo de API

Criar e ativar um ambiente virtual, instalar dependências, fazer uma requisição HTTP, interpretar a resposta JSON e gravá-la em arquivo.

Evidências observáveis

- Explica para que serve um ambiente virtual em uma frase
- Dado um JSON de resposta, acessa um campo aninhado em Python
- Reconhece o significado de um status code 200 e 404

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `PR02.03`

Cursos da trilha

PR02.01 — Python: crie a sua primeira aplicação

8h · 56 atividades · atualizado em 2026-01-26 | Etapa: *Entendendo a linguagem Python*
https://www.alura.com.br/curso-online-python-crie-sua-primeira-aplicacao

Módulos da ementa. Manipulação de Strings; Primeiro programa; Boas práticas; Interpolação de String; Módulos e funções; Lista, laços e exceções; Dicionários; Consolidando os conhecimentos.

Conceitos-chave. variável, string, f-string, input, print, if/elif/else, for, while, try/except, lista, dicionário, função, import, PEP 8.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR02.01.O1	Criar e executar um projeto Python no VSCode, escrevendo um programa que lê e exibe dados	aplicar	PR02.C1
PR02.01.O2	Formatar mensagens com interpolação de strings (f-strings) e manipular textos	aplicar	PR02.C1
PR02.01.O3	Controlar o fluxo de uma aplicação com condicionais e laços de repetição	aplicar	PR02.C2
PR02.01.O4	Proteger a execução de erros previsíveis com blocos try-except	aplicar	PR02.C2
PR02.01.O5	Registrar e consultar dados em listas e dicionários	aplicar	PR02.C3
PR02.01.O6	Criar funções para organizar o menu principal e as operações do programa	aplicar	PR02.C4
PR02.01.O7	Seguir as convenções de nomenclatura e formatação da comunidade Python	entender	PR02.C1

Competências alimentadas por este curso: PR02.C1 Escrever e executar um programa Python usando variáveis, strings e entrada/saída; PR02.C2 Controlar o fluxo do programa com condicionais, laços e tratamento de exceções; PR02.C3 Armazenar e percorrer dados em listas e dicionários; PR02.C4 Organizar o código em funções e módulos.

Fora de escopo (o questionário não cobra). decorators próprios; generators; async/await; type hints avançados; testes automatizados.

PR02.02 — Python: aplicando a Orientação a Objetos

6h · 42 atividades · atualizado em 2025-12-18 | Etapa: *Orientação a objetos*
https://www.alura.com.br/curso-online-python-aplicando-orientacao-objetos

Módulos da ementa. Classes; Atributos de instância; Construtor e instanciando objetos; Property e métodos de classe; Importando classe e composição; Consolidando os conhecimentos.

Conceitos-chave. classe, objeto, instância, atributo, método, self, __init__, @property, setter, @classmethod, @staticmethod, encapsulamento, composição.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR02.02.O1	Explicar por que a Orientação a Objetos organiza melhor um programa que cresce	entender	PR02.C5

Objetivo	Enunciado	Bloom	Competência
PR02.02.O2	Declarar uma classe com seus atributos de instância a partir de um enunciado do mundo real	aplicar	PR02.C5
PR02.02.O3	Usar o construtor ___init___ para inicializar o estado de cada objeto	aplicar	PR02.C5
PR02.02.O4	Expor atributos com @property, inserindo lógica de leitura e de validação na escrita	aplicar	PR02.C6
PR02.02.O5	Distinguir métodos de instância, de classe e estáticos pelo que cada um precisa acessar	analisar	PR02.C6
PR02.02.O6	Importar uma classe de outro arquivo e compor objetos que contêm outros objetos	aplicar	PR02.C4

Competências alimentadas por este curso: PR02.C4 Organizar o código em funções e módulos; PR02.C5 Modelar um domínio com classes, atributos de instância e construtor; PR02.C6 Aplicar encapsulamento com property e métodos de classe e estáticos.

Fora de escopo (o questionário não cobra). metaclasses; dataclasses; mixins; SOLID; design patterns.

PR02.03 — Python: avance na Orientação a Objetos e consuma API

8h · 47 atividades · atualizado em 2025-12-22 | Etapa: *Ambientes virtuais, arquivos e APIs*
https://www.alura.com.br/curso-online-python-avance-orientacao-objetos-consuma-api

Módulos da ementa. Herança; Criando a classe ItemCardapio; Acessando os itens do cardápio; Polimorfismo e método abstrato; Ambientes virtuais; Requisições, JSON e arquivos; FastAPI.

Conceitos-chave. herança, superclasse, subclasse, super(), polimorfismo, classe abstrata, ABC, venv, pip, requisição HTTP, JSON, API, FastAPI.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR02.03.O1	Criar uma subclasse que herda atributos e métodos de uma superclasse	aplicar	PR02.C7
PR02.03.O2	Sobrescrever métodos para obter comportamentos diferentes sob a mesma chamada (polimorfismo)	analisar	PR02.C7
PR02.03.O3	Reconhecer o papel de uma classe abstrata como contrato que as subclasses devem cumprir	entender	PR02.C7
PR02.03.O4	Criar e ativar um ambiente virtual e instalar dependências do projeto	aplicar	PR02.C8
PR02.03.O5	Consumir dados de uma API externa e interpretar a resposta em JSON	aplicar	PR02.C8
PR02.03.O6	Gravar e ler dados de um arquivo JSON a partir de objetos Python	aplicar	PR02.C8

Competências alimentadas por este curso: PR02.C7 Reutilizar código com herança, polimorfismo e classes abstratas; PR02.C8 Trabalhar com ambiente virtual, arquivos JSON e consumo de API.

Fora de escopo (o questionário não cobra). autenticação OAuth; banco de dados; deploy; testes

de API; assincronismo no FastAPI.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece a sintaxe mas não consegue prever o comportamento de um programa; não modela classes sozinho.
basico	50–69	Lê e completa código procedural simples; entende classe e objeto no discurso, mas erra ao aplicar construtor e self.
adequado	70–84	Escreve programas com coleções, funções e tratamento de erro; modela uma classe com construtor e atributos a partir de um enunciado e usa herança básica.
avancado	85–100	Escolhe entre herança e composição com justificativa, usa encapsulamento com property e integra dados externos via API/JSON.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 45 minutos estimados. Por tipo na matriz: multipla_escolha 11 · analise_codigo 7 · dissertativa 2. Por nível cognitivo: lembrar/entender 5 · aplicar 11 · analisar/avaliar 4.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 Variáveis, strings, entrada/saída	2	Múltipla escolha	0	f-string, input() devolve string
C2 Condicionais, laços, exceções	3	Múltipla escolha	1	for vs while, range() exclusivo, qual except captura
C3 Listas e dicionários	2	Múltipla escolha	1	valor aninhado, append() e len(), for sobre a coleção
C3 Listas e dicionários	1	Múltipla resposta	0	quais afirmações sobre lista e dicionário estão corretas
C4 Funções e módulos	1	Múltipla escolha	1	parâmetro vs argumento, import de outro arquivo
C4 Funções e módulos	1	Dissertativa	0	por que print() não substitui return
C5 Classes, atributos, construtor	4	Múltipla escolha	2	___init___ e self, classe vs instância, atributos

Tema	Qtd.	Botão	Com código	Foco da linha
C6 Property e métodos de classe	2	Múltipla escolha	1	leitura com @property, papel do setter, underscore
C7 Herança e polimorfismo	1	Múltipla escolha	1	super().__init__(), qual método roda
C7 Herança e polimorfismo	1	Dissertativa	0	escolher entre herança e composição, com justificativa
C8 Ambiente virtual, JSON e API	1	Múltipla escolha	0	para que serve o venv, status 200 e 404
C8 Ambiente virtual, JSON e API	1	Múltipla resposta	0	quais passos corretos para consumir uma API e ler o JSON

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 7 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/PR02-python-orientacao-objetos.json‘ (‘multipla_escolha‘ 11 + ‘analise_codigo‘ 7) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 7 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - PYTHON COM ORIENTAÇÃO A OBJETOS

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. O estudante já passou por
uma trilha introdutória de lógica em JavaScript e agora programa em Python 3. O projeto
fio condutor da trilha é um sistema de cadastro e avaliação de restaurantes e cardápio.

ASSUNTOS

C1 - Programa Python, variáveis e strings
  Cobrar: prever a saída de um trecho com variáveis e print(); montar uma f-string que
  interpola o valor de uma variável; reconhecer que input() devolve string e precisa de
  conversão para cálculo numérico.
  Erros típicos, para virar alternativa errada: somar o retorno de input() como se fosse
  número; confundir = com ==; esquecer o f no início da f-string; errar a indentação
  dentro de um bloco.

C2 - Condicionais, laços e tratamento de exceções
  Cobrar: escolher entre for e while justificando pela condição de parada; prever quantas
  vezes um laço executa; identificar qual except captura um erro específico (ValueError,
  KeyError).
  Erros típicos: condição de parada que gera laço infinito; erro de 1 na contagem de
  range(), esquecendo que o fim é exclusivo; except genérico que esconde o erro real;
  código dependente do laço escrito fora da indentação dele.

C3 - Listas e dicionários
  Cobrar: justificar dicionário quando o dado tem rótulo e lista quando a ordem importa;
  acessar um valor aninhado em uma lista de dicionários; prever o resultado de append(),
  len() e de um for sobre a coleção.
```

```
  Erros típicos: achar que a lista começa no índice 1; acessar chave inexistente sem
  tratar KeyError; confundir a sintaxe de lista [] com a de dicionário {}; modificar a
  lista enquanto a percorre.

C4 - Funções e módulos
  Cobrar: distinguir parâmetro de argumento e return de print; identificar qual trecho
  repetido deveria virar função; escrever o import correto para usar uma classe ou função
  de outro arquivo.
  Erros típicos: usar print() no lugar de return e depois tentar aproveitar o valor;
  definir a função e esquecer de chamá-la; confundir variável local com global.

C5 - Classes, atributos de instância e construtor
  Cobrar: dado "um jogo tem nome, nota e gênero", escrever a classe com __init__ e os três
  atributos; explicar o papel do self no construtor e nos métodos; distinguir a classe
  (o molde) do objeto (a instância).
  Erros típicos: esquecer o self na assinatura do método; confundir atributo de classe com
  atributo de instância; chamar __init__ diretamente em vez de instanciar a classe; achar
  que todos os objetos compartilham os mesmos valores de atributo.

C6 - Encapsulamento com property e métodos de classe e estáticos
  Cobrar: reconhecer que @property permite rodar lógica (validação, formatação) ao ler um
  atributo; identificar a convenção do underscore (_atributo) como sinal de uso interno;
  saber que é o setter quem valida o valor antes de atribuir.
  Erros típicos: chamar a property com parênteses, como se fosse método; criar um setter
  com nome diferente do getter; achar que o underscore impede o acesso pela linguagem.

C7 - Herança, polimorfismo e classes abstratas
  Cobrar: dado um conjunto de classes, apontar qual deveria ser a superclasse e por quê;
  prever qual implementação de método roda em uma chamada polimórfica; explicar por que
  uma classe abstrata não pode ser instanciada.
  Erros típicos: esquecer super().__init__() na subclasse e perder os atributos herdados;
  confundir sobrescrever um método com criar outro de nome parecido; usar herança onde
  composição seria mais adequada.

C8 - Ambiente virtual, arquivos JSON e consumo de API
  Cobrar: explicar para que serve um ambiente virtual em uma frase; acessar um campo
  aninhado de um JSON de resposta em Python; reconhecer o significado dos status 200 e 404.
  Erros típicos: instalar a dependência sem ativar o ambiente virtual; tratar a resposta
  da API como texto quando precisa de .json(); confundir o arquivo JSON com um dicionário
  Python já pronto.

FORA DE ESCOPO - não cobre nada disso: decorators próprios, generators, async/await, type
hints avançados, testes automatizados, metaclasses, dataclasses, mixins, SOLID, design
patterns, autenticação OAuth, banco de dados, deploy e assincronismo no FastAPI.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: jogos, música, escola, time, rede social,
  lanchonete.
- Quando o item trouxer código, use Python 3 com no máximo 15 linhas e só a biblioteca
  padrão - a única exceção é requests, nos itens de C8.
- Nada de pegadinha de sintaxe fora do que a trilha ensinou. Se o erro a ser detectado for
  de digitação e não de conceito, o item não serve.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie conhecimento de programação, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 5 itens de compreensão, 11 de aplicação e 4
```

```
de análise ou avaliação.
```

PR03 — Django: crie aplicações em Python

Escola (ITT)	Programação
Nome na planilha ITT	Django: crie aplicações em Python
Nome atual na Alura	Django: crie aplicações em Python
Página da formação	`https://www.alura.com.br/formacao-django`
Carga horária	44h em 5 cursos
Pré-requisitos	PR01, PR02
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, já alfabetizados em Python e Orientação a Objetos (PR01 e PR02), iniciando em desenvolvimento web com Python
Projeto fio condutor	Galeria de fotografias (projeto Galeria/Alura Space) construída do zero: templates, banco de dados, área administrativa, cadastro e login de usuários, CRUD completo e imagens persistidas na nuvem
Matriz	`cursos/PR03-django-aplicacoes-python.json`
Prompt	`prompts/PR03.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`PR03.C1`	Estruturar um projeto Django e mapear URLs para views	0.14	3	aplicar
`PR03.C2`	Renderizar páginas com templates sem duplicar código	0.14	3	aplicar
`PR03.C3`	Modelar e persistir dados com models e migrações	0.15	3	aplicar
`PR03.C4`	Administrar os dados pelo Django Admin	0.10	2	aplicar
`PR03.C5`	Construir e validar formulários	0.15	3	aplicar
`PR03.C6`	Autenticar e autorizar pessoas usuárias	0.15	3	analisar
`PR03.C7`	Implementar o CRUD completo no fluxo MTV	0.10	2	aplicar
`PR03.C8`	Lidar com arquivos de mídia e armazenamento externo	0.07	1	entender
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

PR03.C1 — Estruturar um projeto Django e mapear URLs para views

Criar o projeto e as apps, subir o servidor de desenvolvimento, distinguir o papel de projeto, app, view e rota, e manter as configurações sensíveis em variáveis de ambiente.

Evidências observáveis

- Dada uma URL que o navegador precisa atender, aponta qual arquivo de rotas e qual view respondem por ela
- Explica a diferença entre o projeto e uma app dentro dele
- Reconhece que a SECRET_KEY e as credenciais vão para variáveis de ambiente, não para o código versionado

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR03.01`

PR03.C2 — Renderizar páginas com templates sem duplicar código

Carregar um template a partir de uma view, servir arquivos estáticos e imagens, e reaproveitar estrutura com herança de template, partials e url name em vez de caminhos fixos.

Evidências observáveis

- Dada uma view, escreve o render() que devolve o template com o contexto certo
- Usa {% extends %} e {% block %} para não repetir cabeçalho e rodapé em cada página
- Troca um caminho fixo no href pelo {% url %} com o nome da rota

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR03.01`, `PR03.03`

PR03.C3 — Modelar e persistir dados com models e migrações

Declarar um model com os campos adequados ao dado, gerar e aplicar a migração, criar e consultar registros, e filtrar a listagem para implementar uma busca.

Evidências observáveis

- Dado 'uma fotografia tem nome, legenda, data e foto', escreve o model com os campos de tipo correto
- Sabe que alterar o model exige makemigrations e migrate para o banco acompanhar
- Escreve o filter() que devolve só os registros que casam com o termo buscado

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR03.02`

PR03.C4 — Administrar os dados pelo Django Admin

Registrar um model no Admin, operar o CRUD por essa área e personalizar a listagem com os campos e as ações que a pessoa administradora precisa.

Evidências observáveis

- Reconhece que um model só aparece no Admin depois de registrado
- Identifica o que muda na tela do Admin ao declarar list_display e list_filter
- Explica por que o Admin é para quem administra o site, não para quem visita

Tipos de item: `multipla_escolha` **Cursos de origem:** `PR03.02`

PR03.C5 — Construir e validar formulários

Declarar um formulário com as classes do Django, exibi-lo no template, validar os dados recebidos e devolver mensagens de erro e de sucesso compreensíveis.

Evidências observáveis

- Dado um formulário de cadastro, aponta onde a validação acontece antes de salvar
- Sabe que is_valid() precisa rodar antes de confiar em cleaned_data
- Usa o framework de messages para avisar a pessoa do erro em vez de falhar em silêncio

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR03.03`

PR03.C6 — Autenticar e autorizar pessoas usuárias

Implementar cadastro, login e logout com o sistema de usuários do Django, restringir páginas a quem está autenticado e reconhecer o fluxo de um login por provedor externo via OAuth 2.0.

Evidências observáveis

- Distingue autenticar (provar quem é) de autorizar (o que pode fazer)
- Sabe qual decorator protege uma view que só pessoas logadas podem abrir
- Explica, no fluxo OAuth 2.0, que a senha é digitada no provedor e não na aplicação

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `PR03.03`, `PR03.04`

PR03.C7 — Implementar o CRUD completo no fluxo MTV

Percorrer o caminho Model-Template-View para criar, listar, editar e deletar um registro, com a rota, a view e o template de cada operação.

Evidências observáveis

- Dado o CRUD de um recurso, nomeia as quatro operações e o que cada view precisa receber
- Identifica que a edição carrega o registro existente no formulário antes de salvar
- Explica o papel de cada letra do MTV no caminho de uma requisição

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR03.05`

PR03.C8 — Lidar com arquivos de mídia e armazenamento externo

Diferenciar arquivo estático de arquivo de mídia, receber o upload de uma imagem, e reconhecer o que muda ao guardar essa imagem em um bucket S3 em vez do disco local.

Evidências observáveis

- Distingue arquivo estático (CSS, logo do site) de arquivo de mídia (foto enviada pela pessoa)
- Explica em uma frase por que guardar a imagem fora do servidor da aplicação
- Reconhece que a chave de acesso ao bucket é credencial e não vai para o repositório

Tipos de item: `multipla_escolha` **Cursos de origem:** `PR03.02`, `PR03.05`

Cursos da trilha

PR03.01 — Django: templates e boas práticas

8h · 49 atividades · atualizado em 2025-11-06 | Etapa: *Boas práticas*
https://www.alura.com.br/curso-online-django-templates-boas-praticas

Módulos da ementa. Iniciando aplicação e subindo o servidor; Configurações, Git e GitHub; Projeto, app e views; Arquivos estáticos; Boas práticas e partials.

Conceitos-chave. Django, virtualenv, servidor de desenvolvimento, projeto, app, view, URLconf, template, arquivos estáticos, variáveis de ambiente, url name, DRY, partial.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR03.01.O1	Criar um projeto Django em ambiente virtual e subir o servidor de desenvolvimento	aplicar	PR03.C1
PR03.01.O2	Separar projeto e app e isolar as rotas de cada app em seu próprio arquivo de URLs	aplicar	PR03.C1
PR03.01.O3	Mover configurações sensíveis para variáveis de ambiente antes de versionar o projeto	aplicar	PR03.C1
PR03.01.O4	Renderizar um template a partir de uma view e servir arquivos estáticos e imagens	aplicar	PR03.C2
PR03.01.O5	Evitar duplicação com herança de template, partials e url name no lugar de caminhos fixos	aplicar	PR03.C2

Competências alimentadas por este curso: PR03.C1 Estruturar um projeto Django e mapear URLs para views; PR03.C2 Renderizar páginas com templates sem duplicar código.

Fora de escopo (o questionário não cobra). class-based views; Django REST Framework; middlewares próprios; cache; internacionalização completa.

PR03.02 — Django: persistência de dados e Admin

10h · 45 atividades · atualizado em 2026-02-06 | Etapa: *Banco de dados*
https://www.alura.com.br/curso-online-django-persistencia-dados-admin

Módulos da ementa. Lidando com dados; Admin; Avançando no admin; Imagens e filtros; Mecanismo de busca.

Conceitos-chave. model, campo, migração, makemigrations, migrate, ORM, QuerySet, filter, Django Admin, superusuário, list_display, ForeignKey, arquivo de mídia, busca.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR03.02.O1	Declarar um model com os campos adequados ao dado e aplicar a migração correspondente no banco	aplicar	PR03.C3
PR03.02.O2	Criar e consultar registros pelo ORM e exibi-los na página	aplicar	PR03.C3
PR03.02.O3	Implementar uma busca filtrando os registros pelo termo informado	aplicar	PR03.C3
PR03.02.O4	Registrar um model no Django Admin e operar o CRUD por essa área	aplicar	PR03.C4

Objetivo	Enunciado	Bloom	Competência
PR03.02.O5	Personalizar a listagem do Admin e adicionar ações como publicar um registro	aplicar	PR03.C4
PR03.02.O6	Receber o upload de uma imagem e configurar o caminho dos arquivos de mídia	aplicar	PR03.C8

Competências alimentadas por este curso: PR03.C3 Modelar e persistir dados com models e migrações; PR03.C4 Administrar os dados pelo Django Admin; PR03.C8 Lidar com arquivos de mídia e armazenamento externo.

Fora de escopo (o questionário não cobra). migrações customizadas; múltiplos bancos; otimização de queries; transações explícitas; PostgreSQL em produção.

PR03.03 — Django: autenticação de formulários e alerta

10h · 44 atividades · atualizado em 2024-01-03 | Etapa: *Segurança*
https://www.alura.com.br/curso-online-django-autenticacao-formularios-alerta

Módulos da ementa. App usuários; Formulário no Django; Autenticação de usuários; Validações; Refatoração.

Conceitos-chave. `forms.Form`, `widget`, `csrf_token`, `is_valid`, `cleaned_data`, `clean_<campo>`, `User`, `authenticate`, `login`, `logout`, `messages framework`, `partial`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR03.03.O1	Declarar um formulário com as classes do Django e exibi-lo estilizado no template	aplicar	PR03.C5
PR03.03.O2	Validar os dados recebidos com clean e devolver mensagens de erro específicas do campo	aplicar	PR03.C5
PR03.03.O3	Exibir alertas de erro e de sucesso com o framework de messages	aplicar	PR03.C5
PR03.03.O4	Implementar cadastro, login e logout usando o modelo de usuário do Django	aplicar	PR03.C6
PR03.03.O5	Reorganizar os templates em partials para não repetir o bloco de alertas	aplicar	PR03.C2

Competências alimentadas por este curso: PR03.C2 Renderizar páginas com templates sem duplicar código; PR03.C5 Construir e validar formulários; PR03.C6 Autenticar e autorizar pessoas usuárias.

Fora de escopo (o questionário não cobra). recuperação de senha por e-mail; permissões e grupos avançados; autenticação em dois fatores; modelo de usuário customizado.

PR03.04 — Django: autenticação com OAuth2.0

8h · 40 atividades · atualizado em 2024-07-18 | Etapa: *Segurança*
https://www.alura.com.br/curso-online-django-autenticacao-oauth2-0

Módulos da ementa. Servidor; OAuth 2.0; Admin, Github e OAuth; Apenas para pessoas autenticadas; OAuth com GitHub.

Conceitos-chave. `OAuth 2.0`, `provedor de identidade`, `django-allauth`, `registro da aplicação`, `decorator`, `login_required`, `redirect`, `área de membros`, `STATICFILES`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
`PR03.04.O1`	Explicar o fluxo do OAuth 2.0 e o papel do provedor externo no login	entender	`PR03.C6`
`PR03.04.O2`	Configurar o django-allauth e registrar a aplicação no provedor para habilitar o login pelo GitHub	aplicar	`PR03.C6`
`PR03.04.O3`	Restringir uma área da aplicação a pessoas autenticadas usando decorators	aplicar	`PR03.C6`

Competências alimentadas por este curso: `PR03.C6` Autenticar e autorizar pessoas usuárias.

Fora de escopo (o questionário não cobra). escrever um servidor de autorização; JWT e refresh tokens; SAML; SSO corporativo; provedores além do GitHub.

`PR03.05` — Django: CRUD e persistência no S3

8h · 43 atividades · atualizado em 2024-01-03 | Etapa: *Cloud*
`https://www.alura.com.br/curso-online-django-crud-persistencia-s3`

Módulos da ementa. Melhorias no projeto; Inserir fotografias; Deleção, edição e filtro; Lidando com a AWS; Melhorando a persistência.

Conceitos-chave. `CRUD`, `MTV`, `create`, `read`, `update`, `delete`, `filtro`, `AWS`, `bucket S3`, `IAM`, `credencial`, `persistência externa`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
`PR03.05.O1`	Implementar criar, listar, editar e deletar um registro percorrendo o fluxo Model-Template-View	aplicar	`PR03.C7`
`PR03.05.O2`	Aplicar um filtro na listagem para mostrar só os registros de uma categoria	aplicar	`PR03.C7`
`PR03.05.O3`	Criar um bucket no S3 e um usuário na IAM com as permissões necessárias	aplicar	`PR03.C8`
`PR03.05.O4`	Conectar o Django ao bucket para persistir as imagens e manter as credenciais fora do repositório	aplicar	`PR03.C8`

Competências alimentadas por este curso: `PR03.C7` Implementar o CRUD completo no fluxo MTV; `PR03.C8` Lidar com arquivos de mídia e armazenamento externo.

Fora de escopo (o questionário não cobra). CloudFront e CDN; políticas IAM avançadas; infraestrutura como código; deploy da aplicação em produção; custos e billing da AWS.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece o vocabulário do Django, mas não percorre o caminho de uma requisição nem sabe onde mexer para criar uma página nova.
basico	50–69	Cria rota, view e template de uma página simples; erra em migrações, validação de formulário e na diferença entre autenticar e autorizar.
adequado	70–84	Monta uma aplicação com models e migrações, renderiza templates sem duplicar código, valida formulários e implementa cadastro e login com o sistema do Django.
avancado	85–100	Implementa o CRUD completo no fluxo MTV, personaliza o Admin, explica o fluxo OAuth 2.0 e justifica por que credenciais e arquivos de mídia ficam fora do repositório.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Atenção: os cursos 4 e 5 da trilha (OAuth 2.0 e S3) são declarados pela Alura para público profissional e tratam de temas de infraestrutura. A matriz cobra deles apenas o conceito, com peso somado de 0.22, e deixa a operação da AWS fora de escopo. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 45 minutos estimados. Por tipo na matriz: multipla_escolha 11 · analise_codigo 7 · dissertativa 2. Por nível cognitivo: lembrar/entender 5 · aplicar 11 · analisar/avaliar 4.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 Projeto, app, rotas e configuração	2	Múltipla escolha	1	qual rota e qual view atendem a URL, projeto vs app
C1 Projeto, app, rotas e configuração	1	Múltipla resposta	0	quais itens precisam ir para variáveis de ambiente
C2 Templates sem duplicação	3	Múltipla escolha	1	render e contexto, extends e block, tag url
C3 Models e migrações	3	Múltipla escolha	2	campo certo para o dado, makemigrations e migrate, filter
C4 Django Admin	1	Múltipla escolha	0	registrar o model, o que list_display muda na tela
C4 Django Admin	1	Múltipla resposta	0	quais afirmações sobre o Admin estão corretas
C5 Formulários e validação	2	Múltipla escolha	1	onde a validação acontece, is_valid antes de cleaned_data

Tema	Qtd.	Botão	Com código	Foco da linha
C5 Formulários e validação	1	Dissertativa	0	por que não salvar direto o que veio em request.POST
C6 Autenticação e autorização	2	Múltipla escolha	1	decorator que protege a view, o que o fluxo OAuth faz
C6 Autenticação e autorização	1	Dissertativa	0	diferença entre autenticar e autorizar, com exemplo da galeria
C7 CRUD no fluxo MTV	2	Múltipla escolha	1	criar vs editar, redirect depois do POST, papéis do MTV
C8 Mídia e armazenamento externo	1	Múltipla escolha	0	estático vs mídia, a chave do bucket é credencial

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 7 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/PR03-django-aplicacoes-python.json‘ (‘multipla_escolha‘ 11 + ‘analise_codigo‘ 7) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 7 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - DJANGO: APLICAÇÕES WEB EM PYTHON

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. O estudante já programa em
Python com Orientação a Objetos e agora constrói sua primeira aplicação web. O projeto fio
condutor é uma galeria de fotografias: templates, banco de dados, área administrativa,
cadastro e login, CRUD completo e imagens guardadas na nuvem.

ASSUNTOS

C1 - Projeto, app, rotas e configuração
  Cobrar: dada uma URL que o navegador precisa atender, dizer qual arquivo de rotas e qual
  view respondem por ela; diferenciar o projeto da app dentro dele; reconhecer que chave
  secreta e credenciais vão para variáveis de ambiente, não para o código versionado.
  Erros típicos, para virar alternativa errada: confundir projeto com app e pôr tudo em um
  módulo; esquecer de registrar a app em INSTALLED_APPS; deixar todas as URLs no arquivo do
  projeto sem isolar por app; versionar a SECRET_KEY junto com o código.

C2 - Templates sem duplicação
  Cobrar: escrever o render() que devolve o template com o contexto certo; usar extends e
  block para não repetir cabeçalho e rodapé; trocar um caminho fixo no href pela tag url
  com o nome da rota.
  Erros típicos: repetir cabeçalho e rodapé em cada template em vez de herdar de um base;
  esquecer o load static e não entender por que a imagem não aparece; escrever o caminho da
  rota na mão e quebrar tudo quando a URL muda; confundir o nome da variável no contexto
  com o nome dela na view.

C3 - Models e migrações
  Cobrar: dado "uma fotografia tem nome, legenda, data e foto", escrever o model com os
  campos de tipo correto; saber que alterar o model exige makemigrations e migrate;
```

```
  escrever o filter() que devolve só os registros que casam com o termo buscado.
  Erros típicos: alterar o model e esperar que o banco mude sozinho; usar CharField para
  tudo, inclusive data e imagem; criar o objeto e esquecer o save(); comparar com == em vez
  de usar os filtros do ORM.

C4 - Django Admin
  Cobrar: reconhecer que um model só aparece no Admin depois de registrado; identificar o
  que muda na tela ao declarar list_display e list_filter; explicar que o Admin é para quem
  administra o site, não para quem visita.
  Erros típicos: criar o model e procurá-lo no Admin sem tê-lo registrado; achar que o
  Admin é a interface pública da aplicação; confundir o superusuário com um usuário comum
  cadastrado pelo site; mexer direto no banco em vez de usar o Admin ou o ORM.

C5 - Formulários e validação
  Cobrar: apontar, em um formulário de cadastro, onde a validação acontece antes de salvar;
  saber que is_valid() precisa rodar antes de confiar em cleaned_data; usar o framework de
  messages para avisar a pessoa em vez de falhar em silêncio.
  Erros típicos: ler request.POST direto e salvar sem validar; esquecer o csrf_token no
  template e tomar erro no envio; nomear o método de validação fora do padrão clean_<campo>
  e ver a regra ser ignorada; deixar a pessoa sem nenhuma mensagem quando o envio falha.

C6 - Autenticação e autorização
  Cobrar: distinguir autenticar (provar quem é) de autorizar (o que pode fazer); dizer qual
  decorator protege uma view que só pessoas logadas podem abrir; explicar que, no fluxo
  OAuth 2.0, a senha é digitada no provedor e não na aplicação.
  Erros típicos: confundir autenticação com autorização; proteger só o link no menu e
  deixar a URL acessível a quem digita direto; guardar senha em texto puro em vez de usar o
  modelo de usuário do Django; achar que a aplicação recebe a senha no login via GitHub.

C7 - CRUD no fluxo MTV
  Cobrar: nomear as quatro operações do CRUD e o que cada view precisa receber; identificar
  que a edição carrega o registro existente no formulário antes de salvar; explicar o papel
  de cada letra do MTV no caminho de uma requisição.
  Erros típicos: confundir a view de criar com a de editar e perder o registro original;
  deletar sem identificar o registro; esquecer o redirect depois do POST e reenviar o
  formulário ao atualizar a página.

C8 - Mídia e armazenamento externo
  Cobrar: distinguir arquivo estático (CSS, logo do site) de arquivo de mídia (foto enviada
  pela pessoa); explicar em uma frase por que guardar a imagem fora do servidor da
  aplicação; reconhecer que a chave de acesso ao bucket é credencial.
  Erros típicos: tratar a foto enviada pela pessoa como arquivo estático; commitar a chave
  de acesso no repositório; esquecer o MEDIA_URL e não entender por que a imagem não
  carrega; achar que o bucket é um banco de dados.

FORA DE ESCOPO - não cobre nada disso: class-based views, Django REST Framework,
middlewares próprios, cache, internacionalização completa, migrações customizadas,
múltiplos bancos, otimização de queries, recuperação de senha por e-mail, permissões e
grupos avançados, autenticação em dois fatores, escrever um servidor de autorização, JWT,
SAML, políticas IAM avançadas, infraestrutura como código, deploy em produção e custos da
AWS.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: galeria de fotos, blog da escola, catálogo de
  jogos, mural de recados, lista de filmes, site de um evento.
- Quando o item trouxer código, use Python com Django ou a linguagem de template do Django,
  no máximo 15 linhas, sem bibliotecas além das que a trilha usa.
```

```
- Nada de pegadinha de sintaxe fora do que a trilha ensinou. Se o erro a ser detectado for
  de digitação e não de conceito, o item não serve.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie conhecimento de Django, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 5 itens de compreensão, 11 de aplicação e 4
  de análise ou avaliação.
```

PR06 — Desenvolva jogos com Unity

Escola (ITT)	Programação
Nome na planilha ITT	Desenvolva jogos com Unity
Nome atual na Alura	Desenvolva jogos com Unity
Página da formação	`https://www.alura.com.br/formacao-desenvolvendo-jogos-2d-unity`
Carga horária	24h em 2 cursos
Pré-requisitos	nenhum
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, iniciando em desenvolvimento de jogos; a trilha introduz a engine e a programação em C# sem exigir experiência prévia
Projeto fio condutor	Um jogo metroidvania 2D construído do zero ao fim: controles, mecânicas de andar, pular e atacar, sistema de dano, mapa em tilemap, menus, coletáveis, batalha contra o chefão e game loop completo
Matriz	`cursos/PR06-jogos-unity.json`
Prompt	`prompts/PR06.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`PR06.C1`	Montar uma cena 2D na Unity com GameObjects e componentes	0.10	2	entender
`PR06.C2`	Programar comportamento em C# com scripts de MonoBehaviour	0.15	3	aplicar
`PR06.C3`	Capturar a entrada da pessoa jogadora	0.10	2	aplicar
`PR06.C4`	Aplicar física 2D e detectar colisões	0.15	3	aplicar
`PR06.C5`	Animar o personagem com o Animator	0.10	2	aplicar
`PR06.C6`	Construir o nível com Tilemap e implementar o sistema de dano	0.15	3	aplicar
`PR06.C7`	Montar a interface do jogo: menus, UI e coletáveis	0.13	3	aplicar
`PR06.C8`	Fechar o game loop: áudio, vitória, derrota e reinício	0.12	2	analisar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

PR06.C1 — Montar uma cena 2D na Unity com GameObjects e componentes

Navegar na interface da engine e compor a cena, entendendo que um GameObject é um contêiner vazio cujo comportamento vem dos componentes anexados a ele.

Evidências observáveis

- Dada uma necessidade do jogo, diz qual componente precisa ser adicionado ao GameObject
- Distingue a cena, o GameObject e o componente
- Reconhece que um sprite é um asset e o GameObject é o que o coloca na cena

Tipos de item: `multipla_escolha` **Cursos de origem:** `PR06.01`

PR06.C2 — Programar comportamento em C# com scripts de MonoBehaviour

Escrever um script anexado a um GameObject e usar os métodos do ciclo de vida da Unity no momento correto: Start para inicializar e Update para o que roda a cada quadro.

Evidências observáveis

- Dado um comportamento desejado, decide se o código vai em Start ou em Update
- Prevê o efeito de um trecho de C# que altera a posição ou o estado de um GameObject
- Escreve o script que faz o inimigo perseguir a pessoa jogadora

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR06.01`, `PR06.02`

PR06.C3 — Capturar a entrada da pessoa jogadora

Ler comandos de teclado, mouse ou controle pelo sistema de input da Unity e transformá-los em ações do personagem.

Evidências observáveis

- Dado 'o personagem deve pular ao pressionar a tecla', escolhe a leitura de input adequada
- Distingue ler se a tecla está pressionada de ler o momento em que ela foi pressionada
- Reconhece que o input é lido no Update para responder a cada quadro

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR06.01`

PR06.C4 — Aplicar física 2D e detectar colisões

Configurar Rigidbody2D e Collider2D para que o personagem caia, pule e colida, e distinguir colisão de sobreposição por trigger.

Evidências observáveis

- Dado um GameObject que atravessa o chão, aponta qual componente está faltando
- Distingue um collider sólido de um trigger e sabe quando usar cada um
- Explica por que mover o personagem pela física difere de mover pela posição direta

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR06.01`

`PR06.C5` — **Animar o personagem com o Animator**

Criar as animações a partir dos sprites e organizar as transições entre estados — parado, andando, atacando — disparadas por parâmetros.

Evidências observáveis

- Dado um conjunto de estados, aponta qual parâmetro dispara a transição entre eles
- Distingue a animação (o clipe) da máquina de estados que decide qual toca
- Reconhece que o script altera o parâmetro e o Animator decide a transição

Tipos de item: `multipla_escolha` **Cursos de origem:** `PR06.01`, `PR06.02`

`PR06.C6` — **Construir o nível com Tilemap e implementar o sistema de dano**

Montar o mapa do jogo com tilemap e tiles reutilizáveis, e implementar vida, dano e morte do personagem e dos inimigos.

Evidências observáveis

- Explica por que o tilemap é melhor que posicionar centenas de sprites um a um
- Dado um sistema de vida, aponta onde o dano é aplicado e onde a morte é verificada
- Reconhece que o tilemap também precisa de collider para o personagem pisar nele

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR06.01`, `PR06.02`

`PR06.C7` — **Montar a interface do jogo: menus, UI e coletáveis**

Construir o menu inicial e de opções, exibir informações na tela, inserir coletáveis com efeitos visuais de partículas e animar a tela de créditos.

Evidências observáveis

- Dado um botão de menu, identifica o que precisa estar ligado a ele para a ação acontecer
- Explica o papel do sistema de partículas no feedback visual de um coletável
- Distingue o elemento de UI, que acompanha a tela, do GameObject no mundo do jogo

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR06.02`

`PR06.C8` — **Fechar o game loop: áudio, vitória, derrota e reinício**

Adicionar trilha e efeitos sonoros com um audio mixer, implementar as condições de vitória e de derrota e garantir que o jogo reinicie corretamente ao terminar.

Evidências observáveis

- Dada uma condição de fim de jogo, descreve o que precisa acontecer na transição
- Explica para que serve um audio mixer quando há trilha e vários efeitos sonoros
- Reconhece que reiniciar a cena precisa devolver o estado do jogo ao início

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `PR06.01`, `PR06.02`

Cursos da trilha

PR06.01 — Unity: criando um jogo metroidvania 2D

12h | Etapa: *Introdução à criação de Jogos 2D*
https://www.alura.com.br/curso-online-unity-criando-jogo-metroidvania-2d

Módulos da ementa. Interface da Unity e primeiros passos em um jogo 2D; Sistema de input e controles do jogo; Scripts em C# e mecânicas de andar, pular e atacar; Física do jogo e sistema de dano; Assets de arte, animações e mapa; Sistema de som.

Conceitos-chave. Unity, cena, GameObject, componente, Inspector, C#, MonoBehaviour, Start, Update, sistema de input, Rigidbody2D, Collider2D, sprite, Animator, tilemap, AudioSource.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR06.01.O1	Reconhecer os painéis da interface da Unity e compor uma cena 2D com GameObjects e componentes	entender	PR06.C1
PR06.01.O2	Escrever um script em C# anexado a um GameObject para alterar seu comportamento	aplicar	PR06.C2
PR06.01.O3	Ler a entrada da pessoa jogadora pelo sistema de input da Unity para criar os controles	aplicar	PR06.C3
PR06.01.O4	Implementar as mecânicas de andar, pular e atacar combinando input, física e script	aplicar	PR06.C4
PR06.01.O5	Configurar Rigidbody2D e Collider2D para obter a física e as colisões do jogo	aplicar	PR06.C4
PR06.01.O6	Adicionar assets de arte e desenvolver as animações do personagem no Animator	aplicar	PR06.C5
PR06.01.O7	Construir o mapa do jogo utilizando o tilemap da Unity	aplicar	PR06.C6
PR06.01.O8	Implementar um sistema de dano para o personagem e os inimigos	aplicar	PR06.C6
PR06.01.O9	Criar o sistema de som com trilha e efeitos sonoros do jogo	aplicar	PR06.C8

Competências alimentadas por este curso: PR06.C1 Montar uma cena 2D na Unity com GameObjects e componentes; PR06.C2 Programar comportamento em C# com scripts de MonoBehaviour; PR06.C3 Capturar a entrada da pessoa jogadora; PR06.C4 Aplicar física 2D e detectar colisões; PR06.C5 Animar o personagem com o Animator; PR06.C6 Construir o nível com Tilemap e implementar o sistema de dano; PR06.C8 Fechar o game loop: áudio, vitória, derrota e reinício.

Fora de escopo (o questionário não cobra). jogos 3D; multiplayer e rede; shaders customizados; publicação em lojas; Unity DOTS e ECS.

PR06.02 — Unity: criando menus, coletáveis e batalha final em um jogo 2D

12h | Etapa: *Aprimorando o jogo*
https://www.alura.com.br/curso-online-unity-criando-menus-coletaveis-batalha-final-jogo-2d

Módulos da ementa. Menu inicial e menu de opções; Coletáveis e sistema de partículas; Audio mixer e gerenciamento de sons; Batalha contra o chefão e comportamento de chasing; Condições de derrota e vitória; Créditos e game loop.

Conceitos-chave. UI, Canvas, botão, menu de opções, coletável, sistema de partículas, audio mixer, chasing, boss, condição de vitória, condição de derrota, créditos, game loop, reinício de cena.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR06.02.01	Criar o menu inicial e o menu de opções com os elementos de UI da Unity	aplicar	PR06.C7
PR06.02.02	Inserir coletáveis no jogo e enriquecer o feedback visual com o sistema de partículas	aplicar	PR06.C7
PR06.02.03	Criar e animar a tela de créditos do jogo	aplicar	PR06.C7
PR06.02.04	Programar o comportamento de perseguição do inimigo chefão na batalha final	aplicar	PR06.C2
PR06.02.05	Diagnosticar e corrigir erros no comportamento do personagem usando os recursos da engine	analisar	PR06.C2
PR06.02.06	Ajustar as animações e transições de estado dos novos comportamentos do jogo	aplicar	PR06.C5
PR06.02.07	Adicionar um audio mixer para gerenciar os diferentes sons do jogo	aplicar	PR06.C8
PR06.02.08	Implementar as condições de vitória e derrota e um game loop que reinicia o jogo ao final	analisar	PR06.C8
PR06.02.09	Ajustar o nível e os desafios do mapa para acomodar a batalha final	aplicar	PR06.C6

Competências alimentadas por este curso: PR06.C2 Programar comportamento em C# com scripts de MonoBehaviour; PR06.C5 Animar o personagem com o Animator; PR06.C6 Construir o nível com Tilemap e implementar o sistema de dano; PR06.C7 Montar a interface do jogo: menus, UI e coletáveis; PR06.C8 Fechar o game loop: áudio, vitória, derrota e reinício.

Fora de escopo (o questionário não cobra). save e load de progresso em arquivo; inteligência artificial com pathfinding; localização do jogo em outros idiomas; monetização; otimização para dispositivos móveis.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece a interface da Unity, mas não relaciona GameObject, componente e script; não prevê o efeito de um trecho de C#.
basico	50–69	Monta uma cena e segue um script já escrito; erra entre Start e Update, e esquece colliders ao montar o cenário.
adequado	70–84	Programa comportamento em C# no método certo do ciclo de vida, liga input a movimento, configura física e colisões e monta o nível com tilemap e sistema de dano.
avancado	85–100	Diagnostica por que uma mecânica não funciona a partir do código, organiza animações por parâmetros do Animator e fecha o game loop com áudio, vitória, derrota e reinício consistente.

Nível mínimo esperado: adequado.

Observação da coordenação. Faixas definidas pela coordenação ITT, não extraídas da Alura. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 45 minutos estimados. Por tipo na matriz: multipla_escolha 13 · analise_codigo 5 · dissertativa 2. Por nível cognitivo: lembrar/entender 6 · aplicar 11 · analisar/avaliar 3.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 Cena, GameObject e componente	2	Múltipla escolha	0	qual componente falta, asset vs instância na cena
C2 Scripts em C# e ciclo de vida	2	Múltipla escolha	2	Start vs Update, efeito do trecho sobre o GameObject
C2 Scripts em C# e ciclo de vida	1	Múltipla resposta	0	quais afirmações sobre Start, Update e Time.deltaTime estão corretas
C3 Entrada da pessoa jogadora	2	Múltipla escolha	1	tecla pressionada vs momento do toque, onde ler o input
C4 Física 2D e colisões	2	Múltipla escolha	1	Rigidbody2D e Collider2D, collider sólido vs trigger
C4 Física 2D e colisões	1	Dissertativa	0	por que mover pela física difere de mover a posição direto
C5 Animações e Animator	2	Múltipla escolha	0	parâmetro que dispara a transição, clipe vs estado
C6 Tilemap e sistema de dano	2	Múltipla escolha	1	para que serve o tilemap, onde aplicar dano e checar a morte
C6 Tilemap e sistema de dano	1	Múltipla resposta	0	quais itens são necessários para o personagem pisar no tilemap
C7 Menus, UI e coletáveis	3	Múltipla escolha	0	ligar a ação ao botão, UI vs mundo do jogo, partículas
C8 Áudio, vitória, derrota e reinício	1	Múltipla escolha	0	para que serve o audio mixer, o que o reinício precisa zerar
C8 Áudio, vitória, derrota e reinício	1	Dissertativa	0	o que acontece ao reiniciar a cena e por que a pontuação pode persistir

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 5 com código no enunciado.

Equivalência com a matriz: os 18 itens objetivos de ‘cursos/PR06-jogos-unity.json‘ (‘multipla_escolha‘ 13 + ‘analise_codigo‘ 5) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 5 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

AVALIAÇÃO PÓS-TRILHA - DESENVOLVIMENTO DE JOGOS 2D COM UNITY

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina computação para adolescentes do ensino fundamental II e médio. A trilha é a primeira experiência do estudante com uma engine de jogos e com a linguagem C#. O projeto fio condutor é um jogo metroidvania 2D levado do zero ao fim: controles, mecânicas de andar, pular e atacar, sistema de dano, mapa em tilemap, menus, coletáveis, batalha contra o chefão e game loop completo.

ASSUNTOS

C1 - Cena, GameObject e componente
Cobrar: dada uma necessidade do jogo, dizer qual componente precisa ser adicionado ao GameObject; distinguir a cena, o GameObject e o componente; reconhecer que o sprite é um asset e o GameObject é o que o coloca na cena.
Erros típicos, para virar alternativa errada: achar que o GameObject já tem comportamento sem nenhum componente; confundir o asset no projeto com a instância dele na cena; editar valores no Inspector durante o Play e perder as alterações ao parar; confundir a hierarquia da cena com a estrutura de pastas do projeto.

C2 - Scripts em C# e ciclo de vida
Cobrar: dado um comportamento desejado, decidir se o código vai em Start ou em Update; prever o efeito de um trecho de C# que altera a posição ou o estado de um GameObject; escrever o script que faz o inimigo perseguir a pessoa jogadora.
Erros típicos: colocar em Update o que deveria rodar uma vez só, em Start; esquecer que Update roda a cada quadro e multiplicar a velocidade sem usar Time.deltaTime; criar o script e não anexá-lo a nenhum GameObject; dar ao arquivo um nome diferente do nome da classe.

C3 - Entrada da pessoa jogadora
Cobrar: dado "o personagem deve pular ao pressionar a tecla", escolher a leitura de input adequada; distinguir ler se a tecla está pressionada de ler o momento em que ela foi pressionada; reconhecer que o input é lido no Update para responder a cada quadro.
Erros típicos: usar a leitura contínua da tecla para uma ação que deveria acontecer uma vez por toque; ler o input em Start e o controle nunca responder; amarrar a ação a uma tecla fixa em vez de passar pelo mapeamento de input; confundir o eixo horizontal do input com a posição do personagem.

C4 - Física 2D e colisões
Cobrar: dado um GameObject que atravessa o chão, apontar qual componente está faltando; distinguir um collider sólido de um trigger e quando usar cada um; explicar por que mover o personagem pela física difere de mover pela posição direta.
Erros típicos: esperar gravidade sem adicionar Rigidbody2D; adicionar Rigidbody2D e esquecer o Collider2D, e o objeto atravessar o cenário; usar trigger onde era preciso bloquear a passagem; alterar a posição direto no Update e brigar com o motor de física.

C5 - Animações e Animator
Cobrar: dado um conjunto de estados, apontar qual parâmetro dispara a transição entre eles; distinguir a animação (o clipe) da máquina de estados que decide qual toca; reconhecer que o script altera o parâmetro e o Animator decide a transição.
Erros típicos: trocar a animação direto no script em vez de alterar o parâmetro do Animator; criar a transição sem condição e a animação nunca parar no estado certo; confundir o clipe de animação com o estado no Animator; esquecer de desmarcar o loop em uma animação que deve tocar uma vez.

C6 - Tilemap e sistema de dano
Cobrar: explicar por que o tilemap é melhor que posicionar centenas de sprites um a um; dado um sistema de vida, apontar onde o dano é aplicado e onde a morte é verificada; reconhecer que o tilemap também precisa de collider para o personagem pisar nele.
Erros típicos: montar o cenário sprite por sprite em vez de usar o tilemap; esquecer o

```
  Tilemap Collider e o personagem atravessar o chão; aplicar dano a cada quadro de contato e
  matar o personagem instantaneamente; verificar a morte antes de subtrair a vida.

C7 - Menus, UI e coletáveis
  Cobrar: dado um botão de menu, identificar o que precisa estar ligado a ele para a ação
  acontecer; explicar o papel do sistema de partículas no feedback visual de um coletável;
  distinguir o elemento de UI, que acompanha a tela, do GameObject no mundo do jogo.
  Erros típicos: colocar o elemento de UI na cena como sprite comum e ele se mover com a
  câmera; criar o botão e não ligar a ação ao evento de clique; destruir o coletável antes
  de somar a pontuação; confundir o efeito de partículas com uma animação de sprite.

C8 - Áudio, vitória, derrota e reinício
  Cobrar: dada uma condição de fim de jogo, descrever o que precisa acontecer na transição;
  explicar para que serve um audio mixer quando há trilha e vários efeitos sonoros;
  reconhecer que reiniciar a cena precisa devolver o estado do jogo ao início.
  Erros típicos: reiniciar a cena e manter pontuação ou vida do jogo anterior em variáveis
  estáticas; tocar o efeito sonoro a cada quadro em que a condição é verdadeira; controlar o
  volume de cada som separadamente em vez de usar o mixer; verificar a vitória em um lugar
  onde a condição nunca é avaliada.

FORA DE ESCOPO - não cobre nada disso: jogos 3D, multiplayer e rede, shaders customizados,
publicação em lojas, Unity DOTS e ECS, save e load de progresso em arquivo, inteligência
artificial com pathfinding, localização do jogo em outros idiomas, monetização e otimização
para dispositivos móveis.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: jogo de plataforma, jogo de nave, labirinto,
  coleta de moedas, batalha contra chefão, corrida.
- Quando o item trouxer código, use C# para Unity com no máximo 15 linhas, só com a API que
  a trilha usa. Descreva em texto o efeito esperado no jogo - não dependa de o estudante ver
  a cena rodando.
- Nada de pegadinha de sintaxe fora do que a trilha ensinou. Se o erro a ser detectado for
  de digitação e não de conceito, o item não serve.
- Não cobre nome de menu, posição de painel ou atalho da interface da Unity: avalie o
  raciocínio sobre a engine, não a memória da tela.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos.
- Calibragem cognitiva do conjunto: cerca de 6 itens de compreensão, 11 de aplicação e 3
  de análise ou avaliação.
```

PR12 — Boas práticas em Python

Escola (ITT)	Programação
Nome na planilha ITT	Boas práticas em Python
Nome atual na Alura	Boas práticas em Python
Página da formação	`https://www.alura.com.br/formacao-boas-praticas-python`
Carga horária	24h em 3 cursos
Pré-requisitos	PR01, PR02
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT que já programam em Python com Orientação a Objetos (PR01 e PR02) e vão aprender a escrever código legível, bem estruturado e testado
Projeto fio condutor	Refatoração de aplicações Python existentes: uma API de catálogo e recomendações recebe boas práticas e testes, e um sistema de pedidos é reestruturado com SOLID e design patterns
Matriz	`cursos/PR12-boas-praticas-python.json`
Prompt	`prompts/PR12.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`PR12.C1`	Escrever código Python legível seguindo a PEP 8	0.18	4	aplicar
`PR12.C2`	Documentar e tipar o código	0.10	2	aplicar
`PR12.C3`	Organizar o código em módulos e tratar erros de forma explícita	0.14	3	analisar
`PR12.C4`	Aplicar os princípios SOLID	0.20	4	analisar
`PR12.C5`	Reconhecer e aplicar design patterns essenciais	0.15	3	analisar
`PR12.C6`	Escrever testes unitários com Pytest	0.15	3	aplicar
`PR12.C7`	Conduzir o desenvolvimento por testes e interpretar a cobertura	0.08	1	avaliar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

`PR12.C1` — Escrever código Python legível seguindo a PEP 8

Aplicar as convenções de nomenclatura, indentação e formatação da PEP 8, e preferir construções idiomáticas da linguagem, como list comprehension, a soluções verbosas.

Evidências observáveis

- Dado um trecho fora do padrão, aponta qual convenção da PEP 8 foi violada e como corrigir
- Nomeia variável, função e classe com o estilo que a PEP 8 prescreve para cada um
- Reescreve um laço que só monta uma lista como uma list comprehension

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR12.01`

`PR12.C2` — **Documentar e tipar o código**

Escrever docstrings que digam o propósito, os parâmetros e o retorno de uma função, e anotar as assinaturas com type hints para tornar o contrato explícito.

Evidências observáveis

- Dada uma função sem documentação, escreve a docstring que descreve propósito, parâmetros e retorno
- Anota a assinatura de uma função com os type hints corretos
- Reconhece que o type hint documenta a intenção, mas não é verificado em tempo de execução

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR12.01`

`PR12.C3` — **Organizar o código em módulos e tratar erros de forma explícita**

Separar responsabilidades em arquivos e módulos coerentes, e tratar as falhas previsíveis capturando a exceção específica em vez de silenciar o erro.

Evidências observáveis

- Dado um arquivo que faz de tudo, propõe a separação em módulos por responsabilidade
- Troca um except genérico pela exceção específica que o trecho pode lançar
- Reconhece quando deixar a exceção subir é melhor do que capturá-la ali

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR12.01`

`PR12.C4` — **Aplicar os princípios SOLID**

Reconhecer a violação de cada um dos cinco princípios em um código existente e propor a reestruturação que a resolve.

Evidências observáveis

- Dada uma classe que acumula tarefas, identifica a violação da responsabilidade única e propõe a separação
- Reconhece que estender comportamento alterando um if gigante viola o princípio aberto/fechado
- Aponta quando uma subclasse não pode substituir a superclasse sem quebrar quem a usa

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `PR12.02`

PR12.C5 — Reconhecer e aplicar design patterns essenciais

Identificar qual dos padrões estudados — Factory Method, Template Method, Strategy, Facade e Observer — resolve um problema recorrente apresentado, e reconhecer o padrão em um código já escrito.

Evidências observáveis

- Dado o problema 'preciso criar objetos de tipos diferentes conforme a entrada', escolhe Factory Method
- Reconhece Strategy em um código que troca o algoritmo por injeção de um objeto
- Explica o que Facade simplifica e o que Observer desacopla

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR12.02`

PR12.C6 — Escrever testes unitários com Pytest

Escrever um teste que verifica uma unidade de código com assert, testar o caminho de erro com pytest.raises, e organizar a execução da suíte com markers.

Evidências observáveis

- Dada uma função, escreve o teste que verifica o retorno esperado para uma entrada conhecida
- Usa pytest.raises para verificar que a exceção certa é lançada
- Reconhece o que o nome e a estrutura de um teste precisam comunicar a quem o lê

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `PR12.03`

PR12.C7 — Conduzir o desenvolvimento por testes e interpretar a cobertura

Aplicar o ciclo do TDD — escrever o teste que falha, fazer passar, refatorar — e ler um relatório de cobertura sabendo o que ele mede e o que não mede.

Evidências observáveis

- Coloca na ordem correta os três passos do ciclo do TDD
- Explica por que o primeiro teste precisa falhar antes de o código existir
- Reconhece que 100% de cobertura não significa ausência de bugs

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `PR12.03`

Cursos da trilha

`PR12.01` — Python: aplicando boas práticas com PEP 8

8h · 46 atividades · atualizado em 2024-12-09 | Etapa: *Aplicando PEP8: melhorando a qualidade do código com práticas consistentes*
`https://www.alura.com.br/curso-online-python-aplicando-boas-praticas-pep-8`

Módulos da ementa. Primeiros passos em Boas Práticas; Trabalhando Nomenclaturas, Tipagem e Arquivos; Melhorando a Formatação e Documentação; Tratando Erros e List Comprehensions; Adicionando Testes Automatizados.

Conceitos-chave. `PEP 8`, `convenção de nomenclatura`, `snake_case`, `CamelCase`, `type hint`, `docstring`, `Black Formatter`, `list comprehension`, `tratamento de erros`, `módulo`, `ambiente virtual`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR12.01.O1	Aplicar as convenções de nomenclatura da PEP 8 a variáveis, funções, classes e constantes	aplicar	PR12.C1
PR12.01.O2	Formatar o código conforme a PEP 8, manualmente e com o auxílio do Black	aplicar	PR12.C1
PR12.01.O3	Substituir um laço que apenas constrói uma lista por uma list comprehension	aplicar	PR12.C1
PR12.01.O4	Anotar as assinaturas de funções e métodos com type hints explícitos	aplicar	PR12.C2
PR12.01.O5	Escrever docstrings que descrevam o propósito, os parâmetros e o retorno de uma função	aplicar	PR12.C2
PR12.01.O6	Organizar o projeto separando as responsabilidades em módulos coerentes	analisar	PR12.C3
PR12.01.O7	Tratar erros capturando a exceção específica em vez de silenciar a falha	aplicar	PR12.C3

Competências alimentadas por este curso: PR12.C1 Escrever código Python legível seguindo a PEP 8; PR12.C2 Documentar e tipar o código; PR12.C3 Organizar o código em módulos e tratar erros de forma explícita.

Fora de escopo (o questionário não cobra). construção de APIs com FastAPI; linters além do Black e do ESLint do projeto; tipagem com mypy em modo estrito; pre-commit hooks; CI/CD.

PR12.02 — Python: construindo sistemas eficientes com SOLID e Design Patterns

8h · 47 atividades · atualizado em 2025-01-20 | Etapa: *Dominando SOLID e Design Patterns: construindo soluções robústas*
https://www.alura.com.br/curso-online-python-construindo-sistemas-eficientes-solid-design-patterns

Módulos da ementa. S - Princípio da responsabilidade única; O - Princípio Aberto/Fechado; L - Princípio da substituição de Liskov; I - Princípio da segregação de interfaces; D - Princípio da inversão de dependências.

Conceitos-chave. SOLID, responsabilidade única, aberto/fechado, substituição de Liskov, segregação de interfaces, inversão de dependências, abstração, Factory Method, Template Method, Strategy, Facade, Observer.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR12.02.O1	Identificar uma classe que acumula responsabilidades e separá-la em classes de propósito único	analisar	PR12.C4
PR12.02.O2	Estender o comportamento do sistema sem alterar o código existente, seguindo o princípio aberto/fechado	aplicar	PR12.C4
PR12.02.O3	Avaliar se uma subclasse pode substituir a superclasse sem quebrar quem a utiliza	avaliar	PR12.C4
PR12.02.O4	Segregar interfaces por cliente e inverter dependências fazendo o código depender de abstrações	analisar	PR12.C4

Objetivo	Enunciado	Bloom	Competência
PR12.02.05	Escolher entre Factory Method, Template Method e Strategy o padrão que resolve o problema apresentado	analisar	PR12.C5
PR12.02.06	Aplicar Facade para simplificar o acesso a um subsistema e Observer para notificar mudanças de estado	aplicar	PR12.C5

Competências alimentadas por este curso: PR12.C4 Aplicar os princípios SOLID; PR12.C5 Reconhecer e aplicar design patterns essenciais.

Fora de escopo (o questionário não cobra). catálogo completo do GoF; arquitetura hexagonal; DDD; injeção de dependência por framework; microsserviços.

PR12.03 — Python e TDD: explorando testes unitários

8h · 47 atividades · atualizado em 2024-04-11 | Etapa: *Automatizando Testes em Python: Escrevendo Testes Unitários com Pytest*
https://www.alura.com.br/curso-online-python-tdd-explorando-testes-unitarios

Módulos da ementa. Testes automatizados; Pytest; Test-Driven Development; Exceptions e Marks; Cobertura de testes.

Conceitos-chave. teste automatizado, teste unitário, Pytest, assert, TDD, red-green-refactor, pytest.raises, marker, cobertura de testes, relatório de cobertura.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
PR12.03.01	Explicar o que um teste automatizado garante e o que ele não garante sobre o código	entender	PR12.C6
PR12.03.02	Escrever um teste unitário com Pytest que verifica o retorno de uma função com assert	aplicar	PR12.C6
PR12.03.03	Testar com pytest.raises que um método lança a exceção esperada	aplicar	PR12.C6
PR12.03.04	Organizar a execução da suíte de testes com markers	aplicar	PR12.C6
PR12.03.05	Aplicar o ciclo do TDD escrevendo o teste que falha antes de implementar a funcionalidade	aplicar	PR12.C7
PR12.03.06	Medir a cobertura de testes e interpretar o que o relatório diz sobre a suíte	avaliar	PR12.C7

Competências alimentadas por este curso: PR12.C6 Escrever testes unitários com Pytest; PR12.C7 Conduzir o desenvolvimento por testes e interpretar a cobertura.

Fora de escopo (o questionário não cobra). mocks e fixtures avançadas; testes de integração; testes end-to-end; property-based testing; testes de carga.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Escreve código que funciona, mas não reconhece violações de convenção nem sabe dizer o que um teste verifica.
basico	50–69	Corrige nomenclatura e formatação quando apontadas; repete os nomes dos princípios SOLID sem identificar a violação no código.
adequado	70–84	Escreve código legível e documentado seguindo a PEP 8, trata erros com a exceção específica, identifica a quebra da responsabilidade única e escreve um teste unitário com Pytest.
avancado	85–100	Diagnostica qual princípio SOLID um código viola e propõe a reestruturação, escolhe o design pattern adequado ao problema e usa o ciclo do TDD sabendo o que a cobertura mede.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Atenção: a Alura declara os três cursos para público de nível intermediário, e o curso de PEP 8 pressupõe familiaridade com FastAPI. A matriz cobra os conceitos de boas práticas, SOLID, padrões e testes, e mantém a construção de APIs fora de escopo — os itens devem usar contextos do nível de PR02. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 45 minutos estimados. Por tipo na matriz: multipla_escolha 10 · analise_codigo 8 · dissertativa 2. Por nível cognitivo: lembrar/entender 4 · aplicar 10 · analisar/avaliar 6.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 PEP 8 e código idiomático	3	Múltipla escolha	2	convenção violada, nome por tipo de elemento, list comprehension
C1 PEP 8 e código idiomático	1	Múltipla resposta	0	quais trechos violam a PEP 8
C2 Docstrings e type hints	2	Múltipla escolha	1	o que a docstring precisa dizer, o que o type hint não garante
C3 Módulos e tratamento de erros	3	Múltipla escolha	1	except específico vs genérico, separação por responsabilidade
C4 Princípios SOLID	3	Múltipla escolha	1	qual princípio o código viola e qual correção resolve
C4 Princípios SOLID	1	Dissertativa	0	identificar a violação da responsabilidade única e propor a separação
C5 Design patterns	2	Múltipla escolha	1	qual padrão resolve o problema, reconhecer o padrão no código
C5 Design patterns	1	Múltipla resposta	0	quais afirmações sobre Strategy, Facade e Observer estão corretas

Tema	Qtd.	Botão	Com código	Foco da linha
C6 Testes unitários com Pytest	3	Múltipla escolha	2	assert que verifica o retorno, pytest.raises, teste sem asserção
C7 TDD e cobertura	1	Dissertativa	0	por que o teste precisa falhar primeiro e o que a cobertura não prova

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 8 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/PR12-boas-praticas-python.json‘ (‘multipla_escolha‘ 10 + ‘analise_codigo‘ 8) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 8 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - BOAS PRÁTICAS EM PYTHON

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. O estudante já programa em
Python com Orientação a Objetos e agora aprendeu a escrever código legível, bem estruturado
e testado. A trilha trabalha sobre código já existente: refatorar, reorganizar e cobrir com
testes.

ASSUNTOS

C1 - PEP 8: nomenclatura, formatação e código idiomático
  Cobrar: dado um trecho fora do padrão, apontar qual convenção da PEP 8 foi violada e como
  corrigir; nomear variável, função e classe no estilo que a PEP 8 prescreve para cada um;
  reescrever um laço que só monta uma lista como list comprehension.
  Erros típicos, para virar alternativa errada: nomear classe em snake_case e função em
  CamelCase, trocando as convenções; achar que a PEP 8 é obrigatória para o interpretador e
  que o código quebra sem ela; usar nomes de uma letra em vez de nomes que dizem o que o
  valor é; transformar em list comprehension um laço que faz outras coisas além de montar a
  lista.

C2 - Docstrings e type hints
  Cobrar: dada uma função sem documentação, escrever a docstring que descreve propósito,
  parâmetros e retorno; anotar a assinatura com os type hints corretos; reconhecer que o
  type hint documenta a intenção mas não é verificado em tempo de execução.
  Erros típicos: achar que o type hint impede passar um valor do tipo errado; escrever
  docstring que repete o nome da função sem dizer nada novo; comentar o óbvio linha a linha
  em vez de documentar o contrato da função; confundir comentário com docstring.

C3 - Módulos e tratamento de erros
  Cobrar: dado um arquivo que faz de tudo, propor a separação em módulos por
  responsabilidade; trocar um except genérico pela exceção específica que o trecho pode
  lançar; reconhecer quando deixar a exceção subir é melhor do que capturá-la ali.
  Erros típicos: usar except Exception e esconder o erro real; capturar a exceção e dar
  pass, deixando o programa seguir em estado inválido; criar um módulo utils onde tudo que
  não tem lugar é despejado; confundir importar o módulo com importar o nome de dentro dele.

C4 - Princípios SOLID
  Cobrar: dada uma classe que acumula tarefas, identificar a violação da responsabilidade
  única e propor a separação; reconhecer que estender comportamento alterando um if gigante
  viola o princípio aberto/fechado; apontar quando uma subclasse não pode substituir a
```

```
  superclasse sem quebrar quem a usa.
  Erros típicos: entender responsabilidade única como "um método por classe"; achar que o
  princípio aberto/fechado proíbe qualquer alteração no código; confundir a segregação de
  interfaces com simplesmente criar mais classes; inverter a dependência trocando uma classe
  concreta por outra classe concreta.

C5 - Design patterns essenciais
  Cobrar: dado o problema "preciso criar objetos de tipos diferentes conforme a entrada",
  escolher Factory Method; reconhecer Strategy em um código que troca o algoritmo por
  injeção de um objeto; explicar o que Facade simplifica e o que Observer desacopla.
  Use apenas estes cinco padrões: Factory Method, Template Method, Strategy, Facade e
  Observer.
  Erros típicos: aplicar um padrão onde uma função simples resolveria; confundir Strategy
  com Template Method (injetar o algoritmo vs sobrescrever o passo); achar que Facade
  esconde a complexidade eliminando-a; chamar de Observer qualquer código que usa callbacks.

C6 - Testes unitários com Pytest
  Cobrar: dada uma função, escrever o teste que verifica o retorno esperado para uma entrada
  conhecida; usar pytest.raises para verificar que a exceção certa é lançada; reconhecer o
  que o nome e a estrutura de um teste precisam comunicar a quem o lê.
  Erros típicos: escrever teste que não afirma nada, só executa a função sem assert; testar
  vários comportamentos em um único teste e não saber qual falhou; esperar que o teste
  capture a exceção com try/except em vez de pytest.raises; achar que teste que passa prova
  que o código está correto em todos os casos.

C7 - TDD e cobertura de testes
  Cobrar: colocar na ordem correta os três passos do ciclo do TDD; explicar por que o
  primeiro teste precisa falhar antes de o código existir; reconhecer que 100% de cobertura
  não significa ausência de bugs.
  Erros típicos: escrever o código primeiro e o teste depois, chamando isso de TDD; tratar
  cobertura alta como garantia de qualidade; pular a etapa de refatoração assim que o teste
  passa; excluir da cobertura o código difícil de testar para inflar o número.

FORA DE ESCOPO - não cobre nada disso: construção de APIs com FastAPI, linters além do
Black, tipagem com mypy em modo estrito, pre-commit hooks, CI/CD, o catálogo completo de
padrões do GoF, arquitetura hexagonal, DDD, injeção de dependência por framework,
microsserviços, mocks e fixtures avançadas, testes de integração, testes end-to-end,
property-based testing e testes de carga.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: catálogo de jogos, sistema de pedidos de uma
  lanchonete, biblioteca da escola, lista de músicas, placar de um campeonato.
- Quando o item trouxer código, use Python 3 com no máximo 15 linhas e só a biblioteca
  padrão, mais o pytest nos itens de C6 e C7. Não use FastAPI nem framework web: a trilha
  usa uma API como veículo, mas o que se avalia é a boa prática, não o framework.
- Nada de pegadinha de sintaxe fora do que a trilha ensinou. Se o erro a ser detectado for
  de digitação e não de conceito, o item não serve.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie conhecimento de boas práticas, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 4 itens de compreensão, 10 de aplicação e 6
  de análise ou avaliação.
```

FE01 — A partir do zero: HTML e CSS para projetos web

Escola (ITT)	Front-end
Nome na planilha ITT	A partir do zero: HTML e CSS para projetos web
Nome atual na Alura	HTML e CSS: do zero ao layout responsivo
Página da formação	`https://www.alura.com.br/formacao-html-css`
Carga horária	54h em 4 cursos
Pré-requisitos	nenhum
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, sem experiência prévia em desenvolvimento web
Projeto fio condutor	Sites completos construídos do zero (portfólio, Cinetopia, Jornada Viagens), versionados no GitHub e publicados no GitHub Pages
Matriz	`cursos/FE01-html-css-projetos-web.json`
Prompt	`prompts/FE01.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`FE01.C1`	Explicar como a internet e a web funcionam	0.10	2	entender
`FE01.C2`	Estruturar uma página com HTML semântico	0.18	4	aplicar
`FE01.C3`	Estilizar com CSS usando seletores e propriedades	0.18	4	aplicar
`FE01.C4`	Controlar espaçamento e dimensões com o box model, e organizar o código	0.10	2	aplicar
`FE01.C5`	Construir formulários utilizáveis e validados	0.12	2	aplicar
`FE01.C6`	Aplicar acessibilidade digital e SEO técnico	0.10	2	aplicar
`FE01.C7`	Montar layouts com Flexbox e CSS Grid	0.12	2	aplicar
`FE01.C8`	Tornar o site responsivo com mobile first e media queries	0.10	2	aplicar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

`FE01.C1` — Explicar como a internet e a web funcionam

Descrever o caminho de uma requisição entre cliente e servidor, reconhecer o papel de HTTP, DNS e TCP/IP e situar o navegador nesse fluxo.

Evidências observáveis

- Coloca em ordem os passos entre digitar um endereço e a página aparecer
- Explica o que o DNS faz em uma frase
- Distingue o papel do cliente e o do servidor em um exemplo concreto

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `FE01.01`

`FE01.C2` — **Estruturar uma página com HTML semântico**

Montar a estrutura de um documento HTML, escolher a tag pelo significado do conteúdo (header, nav, main, section, article, footer), respeitar a hierarquia de títulos e usar atributos essenciais.

Evidências observáveis

- Dado um conteúdo descrito em texto, escolhe a tag semântica adequada
- Corrige uma hierarquia de títulos que pula de h1 para h3
- Identifica o atributo alt como requisito de uma imagem

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.02`, `FE01.04`

`FE01.C3` — **Estilizar com CSS usando seletores e propriedades**

Conectar CSS ao HTML, selecionar elementos por tag, classe e id, aplicar cores, tipografia e espaçamento, e prever qual regra vence quando há conflito.

Evidências observáveis

- Dado um HTML e um CSS, aponta quais elementos recebem determinado estilo
- Escolhe entre classe e id conforme o estilo se repetir ou não
- Resolve um conflito de especificidade simples entre duas regras

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.02`, `FE01.04`

`FE01.C4` — **Controlar espaçamento e dimensões com o box model, e organizar o código**

Distinguir margin, border e padding, prever o tamanho final de uma caixa e manter CSS organizado e reaproveitável.

Evidências observáveis

- Dado um elemento com width, padding e border, identifica o espaço ocupado
- Escolhe entre margin e padding para resolver um problema de espaçamento descrito
- Reconhece quando criar uma classe reutilizável em vez de repetir regras

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.02`

`FE01.C5` — **Construir formulários utilizáveis e validados**

Escolher o tipo de campo adequado ao dado, associar label ao campo pelo id, agrupar opções (radio, checkbox, select) e aplicar validação básica.

Evidências observáveis

- Dado um dado a coletar (e-mail, data, arquivo), escolhe o input adequado
- Associa corretamente label e campo via for/id

- Reconhece o papel do atributo required e do name no envio

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.03`

`FE01.C6` — Aplicar acessibilidade digital e SEO técnico

Aplicar diretrizes de WCAG e ARIA, garantir contraste e navegação por teclado, e usar meta tags e estrutura semântica para que o site seja encontrado nas buscas.

Evidências observáveis

- Identifica uma barreira de acessibilidade em um trecho de HTML
- Explica para que serve o texto alternativo de uma imagem
- Reconhece o papel da meta description e do title na busca

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `FE01.03`, `FE01.04`

`FE01.C7` — Montar layouts com Flexbox e CSS Grid

Escolher entre Flexbox (um eixo) e Grid (duas dimensões), alinhar e distribuir elementos e montar estruturas de página completas.

Evidências observáveis

- Dado um layout desenhado, justifica a escolha entre Flexbox e Grid
- Prevê o resultado visual de justify-content e align-items
- Identifica que display: flex vai no contêiner, não nos filhos

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.04`

`FE01.C8` — Tornar o site responsivo com mobile first e media queries

Escrever CSS partindo da tela pequena, criar pontos de quebra com media queries, usar unidades relativas e adaptar imagens a diferentes telas.

Evidências observáveis

- Prevê qual bloco de media query se aplica a uma dada largura de tela
- Explica o princípio mobile first em uma frase
- Escolhe unidade relativa (%, rem, vw) em vez de px fixo quando o layout deve adaptar

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE01.04`

Cursos da trilha

`FE01.01` — Internet: entendendo os fundamentos da web

8h · 47 atividades · atualizado em 2026-06-15 | Etapa: *Entendendo a internet e criando seu primeiro site*
`https://www.alura.com.br/curso-online-internet-entendendo-fundamentos-da-web`

Módulos da ementa. O que é a internet?; Como surgiu a internet e a Arpanet; TCP/IP e a padronização da comunicação; A criação da World Wide Web; A era dos navegadores; Entendendo a relação cliente-servidor; Como o computador entende os dados; Conhecendo as áreas para pessoas desenvolvedoras.

Conceitos-chave. `internet`, `web`, `Arpanet`, `TCP/IP`, `HTTP`, `DNS`, `cliente-servidor`, `navegador`,

cabos submarinos, DevTools.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE01.01.O1	Compreender a evolução histórica e os fundamentos técnicos da internet	entender	FE01.C1
FE01.01.O2	Identificar tipos de redes e elementos da infraestrutura digital	entender	FE01.C1
FE01.01.O3	Reconhecer o papel dos protocolos HTTP, DNS e TCP na comunicação entre dispositivos	entender	FE01.C1
FE01.01.O4	Explicar a arquitetura cliente-servidor em um exemplo concreto de navegação	entender	FE01.C1
FE01.01.O5	Usar DevTools para observar as requisições e a performance de um site	aplicar	FE01.C6

Competências alimentadas por este curso: FE01.C1 Explicar como a internet e a web funcionam; FE01.C6 Aplicar acessibilidade digital e SEO técnico.

Fora de escopo (o questionário não cobra). configuração de servidores; redes corporativas; criptografia; protocolos de roteamento.

FE01.02 — HTML e CSS: ambiente, estrutura e estilo

20h · 59 atividades · atualizado em 2026-07-24 | Etapa: *Entendendo a internet e criando seu primeiro site*
https://www.alura.com.br/curso-online-html-css-ambiente-estrutura-estilo

Módulos da ementa. Configurando o ambiente e criando o primeiro site; Instalando e configurando o VSCode; Google Chrome e as ferramentas de desenvolvedor; Criando seu primeiro site; Hierarquia de títulos e integridade dos dados; Explorando as tags HTML; HTML semântico e boas práticas de layout; Estilizando e adaptando layouts com CSS; Compartilhando o projeto com Git e GitHub; Hospedagem e publicação de sites.

Conceitos-chave. VSCode, DevTools, tag, atributo, hierarquia de títulos, HTML semântico, seletor, classe, id, box model, margin, padding, Git, GitHub Pages, Figma.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE01.02.O1	Compreender os fundamentos do HTML e a semântica das tags	entender	FE01.C2
FE01.02.O2	Estruturar uma página respeitando a hierarquia de títulos e as boas práticas de layout	aplicar	FE01.C2
FE01.02.O3	Aplicar estilos com CSS usando seletores de tag, classe e id	aplicar	FE01.C3
FE01.02.O4	Controlar espaçamento e dimensões dos elementos com o box model	aplicar	FE01.C4
FE01.02.O5	Organizar e estruturar o código para facilitar a manutenção	aplicar	FE01.C4
FE01.02.O6	Explorar as funcionalidades do VSCode e das DevTools para aumentar a produtividade	aplicar	FE01.C3

Objetivo	Enunciado	Bloom	Competência
FE01.02.07	Versionar o projeto com Git e GitHub e publicá-lo com GitHub Pages	aplicar	FE01.C2

Competências alimentadas por este curso: FE01.C2 Estruturar uma página com HTML semântico; FE01.C3 Estilizar com CSS usando seletores e propriedades; FE01.C4 Controlar espaçamento e dimensões com o box model, e organizar o código.

Fora de escopo (o questionário não cobra). JavaScript; pré-processadores (Sass/Less); frameworks CSS; animações complexas.

FE01.03 — HTML e CSS: formulários, SEO e acessibilidade

6h · 42 atividades · atualizado em 2026-09-21 | Etapa: *Formulários acessíveis e visíveis nas buscas*
https://www.alura.com.br/curso-online-html-css-formularios-seo-acessibilidade

Módulos da ementa. Construindo componentes de formulário com HTML; Criando inputs de texto, e-mail, número e arquivo; Criando radio buttons e select; Lidando com checkboxes e usabilidade com o atributo ID; Implementando múltiplas escolhas; Usabilidade e validação do formulário; Aplicando práticas de acessibilidade digital; Aplicando técnicas de Search Engine Optimization (SEO).

Conceitos-chave. form, input, label, for/id, name, required, radio, checkbox, select, validação, WCAG, ARIA, contraste, Lighthouse, WAVE, meta tags, SEO.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE01.03.01	Estruturar formulários HTML utilizando elementos semânticos e o tipo de campo adequado ao dado	aplicar	FE01.C5
FE01.03.02	Validar dados do formulário com foco na experiência da pessoa usuária	aplicar	FE01.C5
FE01.03.03	Aplicar diretrizes de acessibilidade digital (WCAG e ARIA)	aplicar	FE01.C6
FE01.03.04	Avaliar um site com ferramentas como Google Lighthouse e WAVE	analisar	FE01.C6
FE01.03.05	Implementar estratégias de SEO técnico e de conteúdo	aplicar	FE01.C6

Competências alimentadas por este curso: FE01.C5 Construir formulários utilizáveis e validados; FE01.C6 Aplicar acessibilidade digital e SEO técnico.

Fora de escopo (o questionário não cobra). validação com JavaScript; envio ao back-end; auditoria WCAG completa; SEO off-page e link building.

FE01.04 — HTML e CSS: responsividade em layouts

20h · 79 atividades · atualizado em 2026-01-16 | Etapa: *Adaptando seu site para qualquer tela*
https://www.alura.com.br/curso-online-html-css-responsividade-layouts

Módulos da ementa. Entendendo o projeto e configurando o ambiente; Adaptando a interface para diferentes dispositivos; Aplicando mobile first, flexbox e boas práticas; Criando layouts responsivos com imagens; Avançando em técnicas de responsividade e componentização; Aplicando técnicas de CSS Grid; SEO e acessibilidade na prática; Desafio: avançando o projeto.

Conceitos-chave. `mobile first`, `media query`, `breakpoint`, `viewport`, `unidades relativas (%, rem, vw)`, `Flexbox`, `justify-content`, `align-items`, `CSS Grid`, `componentização`, `imagens responsivas`, `otimização`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
`FE01.04.O1`	Compreender a estrutura e a organização de um projeto web	entender	`FE01.C2`
`FE01.04.O2`	Aplicar HTML semântico para criar layouts acessíveis	aplicar	`FE01.C2`
`FE01.04.O3`	Implementar regras de responsividade com CSS avançado e media queries, partindo do mobile first	aplicar	`FE01.C8`
`FE01.04.O4`	Utilizar Flexbox e CSS Grid para desenvolver interfaces flexíveis	aplicar	`FE01.C7`
`FE01.04.O5`	Justificar a escolha entre Flexbox e CSS Grid conforme o layout desejado	analisar	`FE01.C7`
`FE01.04.O6`	Otimizar imagens e meta tags para melhorar a performance e o SEO do site	aplicar	`FE01.C6`
`FE01.04.O7`	Gerenciar versões e atualizar o código com Git e GitHub	aplicar	`FE01.C3`

Competências alimentadas por este curso: `FE01.C2` Estruturar uma página com HTML semântico; `FE01.C3` Estilizar com CSS usando seletores e propriedades; `FE01.C6` Aplicar acessibilidade digital e SEO técnico; `FE01.C7` Montar layouts com Flexbox e CSS Grid; `FE01.C8` Tornar o site responsivo com mobile first e media queries.

Fora de escopo (o questionário não cobra). JavaScript; frameworks CSS; container queries; design system completo.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece tags isoladas, mas não monta uma página estruturada nem prevê o efeito de uma regra CSS.
basico	50–69	Escreve HTML simples e aplica cores e fontes; erra em semântica, especificidade de seletores e box model.
adequado	70–84	Estrutura páginas com HTML semântico, estiliza com seletores adequados, monta formulários com label e constrói layout com Flexbox e media queries.
avancado	85–100	Justifica Flexbox vs Grid, resolve conflitos de especificidade, aplica acessibilidade e SEO sem consultar e diagnostica por que um layout quebra no celular.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 40 minutos estimados. Por tipo na matriz: multipla_escolha 12 · analise_codigo 6 · dissertativa 2. Por nível cognitivo: lembrar/entender 7 · aplicar 10 · analisar/avaliar 3.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 Internet e web	1	Múltipla escolha	0	papel do DNS, cliente e servidor
C1 Internet e web	1	Dissertativa	0	explicar o caminho entre digitar o endereço e a página aparecer
C2 HTML semântico	4	Múltipla escolha	1	escolha da tag, hierarquia de títulos, alt da imagem
C3 Seletores e propriedades CSS	4	Múltipla escolha	2	classe vs id, quem recebe o estilo, especificidade
C4 Box model e organização	2	Múltipla escolha	1	espaço ocupado, margin vs padding, box-sizing
C5 Formulários	1	Múltipla escolha	0	escolher o input certo para o dado a coletar
C5 Formulários	1	Múltipla resposta	0	quais atributos o campo precisa ter (name, for/id, required)
C6 Acessibilidade e SEO	1	Múltipla escolha	0	meta description e title, barreira de acessibilidade
C6 Acessibilidade e SEO	1	Dissertativa	0	por que o texto alternativo importa e a quem atende
C7 Flexbox e Grid	2	Múltipla escolha	1	contêiner vs filhos, justify-content e align-items
C8 Mobile first e media queries	1	Múltipla escolha	1	qual media query se aplica a uma dada largura
C8 Mobile first e media queries	1	Múltipla resposta	0	quais práticas seguem mobile first de fato

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 6 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/FE01-html-css-projetos-web.json‘ (‘multipla_escolha‘ 12 + ‘analise_codigo‘ 6) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 6 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - HTML E CSS PARA PROJETOS WEB

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. O estudante não tinha
```

```
experiência em desenvolvimento web antes da trilha e construiu sites completos publicados
no GitHub Pages. A trilha não cobre JavaScript - não use JavaScript em nenhum item.

ASSUNTOS

C1 - Como a internet e a web funcionam
  Cobrar: colocar em ordem os passos entre digitar um endereço e a página aparecer; o que
  o DNS faz, em uma frase; o papel do cliente e o do servidor em um exemplo concreto.
  Erros típicos, para virar alternativa errada: achar que internet e web são a mesma
  coisa; supor que o site inteiro fica salvo no navegador; confundir o DNS com o servidor
  que hospeda o site; achar que o Wi-Fi é a internet.

C2 - HTML semântico
  Cobrar: dado um conteúdo descrito em texto, escolher a tag semântica adequada (header,
  nav, main, section, article, footer); corrigir uma hierarquia de títulos que pula de h1
  para h3; reconhecer o alt como requisito de uma imagem.
  Erros típicos: usar div para tudo em vez da tag semântica correta; escolher a tag de
  título pelo tamanho visual e não pela hierarquia; esquecer de fechar tags ou aninhá-las
  na ordem errada; omitir o alt das imagens.

C3 - Seletores e propriedades CSS
  Cobrar: dado um HTML e um CSS, apontar quais elementos recebem determinado estilo;
  escolher entre classe e id conforme o estilo se repita ou não; resolver um conflito
  simples de especificidade entre duas regras.
  Erros típicos: confundir o seletor de classe (.) com o de id (#); repetir o mesmo id em
  vários elementos; esperar que a última regra do arquivo sempre vença, ignorando
  especificidade; esquecer a unidade (px, rem) no valor.

C4 - Box model e organização do código
  Cobrar: dado um elemento com width, padding e border, identificar o espaço que ele
  ocupa; escolher entre margin e padding para resolver um espaçamento descrito; reconhecer
  quando criar uma classe reutilizável em vez de repetir regras.
  Erros típicos: trocar margin por padding e vice-versa; ignorar o efeito de box-sizing no
  cálculo da largura; resolver espaçamento com vários <br> em vez de CSS; escrever estilo
  inline em vez de usar o arquivo CSS.

C5 - Formulários utilizáveis e validados
  Cobrar: dado um dado a coletar (e-mail, data, arquivo), escolher o input adequado;
  associar label e campo via for/id; reconhecer o papel de required e de name no envio.
  Erros típicos: usar input type="text" para tudo; esquecer o name e perder o dado no
  envio; agrupar radios sem o mesmo name, permitindo marcar mais de um; usar placeholder
  como se fosse label.

C6 - Acessibilidade digital e SEO técnico
  Cobrar: identificar uma barreira de acessibilidade em um trecho de HTML; explicar para
  que serve o texto alternativo de uma imagem; reconhecer o papel da meta description e do
  title na busca.
  Erros típicos: achar que acessibilidade é só para pessoas cegas; usar aria-label onde o
  HTML semântico já resolveria; repetir palavras-chave achando que melhora o SEO;
  confundir o title da aba com o h1 da página.

C7 - Layouts com Flexbox e CSS Grid
  Cobrar: dado um layout desenhado, justificar a escolha entre Flexbox e Grid; prever o
  resultado visual de justify-content e align-items; reconhecer que display: flex vai no
  contêiner, não nos filhos.
  Erros típicos: aplicar as propriedades de flex nos itens em vez do contêiner; confundir
  o eixo principal com o transversal; usar Grid para alinhar um único elemento; esquecer
  de definir flex-direction e estranhar o resultado.

C8 - Responsividade com mobile first e media queries
```

> Cobrar: prever qual bloco de media query se aplica a uma dada largura de tela; explicar o princípio mobile first em uma frase; escolher unidade relativa (%, rem, vw) em vez de px fixo quando o layout deve adaptar.
> Erros típicos: escrever media query com min-width achando que vale para telas menores; definir larguras fixas em px e quebrar o layout no celular; esquecer a meta tag viewport; criar um breakpoint por aparelho em vez de por conteúdo.

FORA DE ESCOPO - não cobre nada disso: JavaScript em qualquer forma, pré-processadores (Sass, Less), frameworks CSS, animações complexas, container queries, design system completo, validação via script, envio ao back-end, auditoria WCAG completa, SEO off-page e link building, configuração de servidores e criptografia.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo 3 linhas.
- Contextos concretos e próximos do público: página de um jogo, perfil, blog da escola, cardápio, portfólio, site de um evento.
- Quando o item trouxer código, use HTML e CSS com no máximo 15 linhas, sem JavaScript, sem frameworks e sem pré-processadores. Descreva em texto o efeito visual esperado: o estudante não vai ver a página renderizada.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou ordem dos módulos. Avalie conhecimento de HTML e CSS, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 7 itens de compreensão, 10 de aplicação e 3 de análise ou avaliação.

FE03 — Desenvolvimento Front-end: criar aplicações web com HTML, CSS e JavaScript

Escola (ITT)	Front-end
Nome na planilha ITT	Desenvolvimento Front-end: criar aplicações web com HTML, CSS e JavaScript
Nome atual na Alura	Desenvolvimento Front-end: cursos para criar aplicações web com HTML, CSS e JavaScript
Página da formação	`https://www.alura.com.br/formacao-front-end`
Carga horária	60h em 8 cursos
Pré-requisitos	FE01
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT que já constroem páginas com HTML e CSS (FE01) e vão aprender a torná-las dinâmicas com JavaScript
Projeto fio condutor	Aplicações web interativas construídas do zero ao deploy: lista de tarefas, temporizador Pomodoro (Fokus), mural de pensamentos com CRUD em API e catálogo de livros com filtros
Matriz	`cursos/FE03-front-end-javascript.json`
Prompt	`prompts/FE03.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`FE03.C1`	Integrar JavaScript à página, selecionar elementos e depurar no navegador	0.10	2	aplicar
`FE03.C2`	Reagir a eventos da pessoa usuária	0.10	2	aplicar
`FE03.C3`	Criar, alterar e remover elementos e estilos no DOM	0.15	3	aplicar
`FE03.C4`	Processar coleções com os métodos de array	0.14	3	aplicar
`FE03.C5`	Lidar com código assíncrono	0.14	3	analisar
`FE03.C6`	Consumir uma API com fetch e implementar o CRUD via HTTP	0.15	3	aplicar
`FE03.C7`	Persistir e validar dados no navegador	0.12	2	aplicar
`FE03.C8`	Modularizar o código e usar o ambiente Node e NPM	0.10	2	entender
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

FE03.C1 — Integrar JavaScript à página, selecionar elementos e depurar no navegador

Ligar o script ao HTML, entender o DOM como a representação da página em objetos, selecionar elementos por seletor e investigar o comportamento do código nas ferramentas do navegador.

Evidências observáveis

- Dado um elemento no HTML, escreve o seletor que o recupera em JavaScript
- Explica o que é o DOM e por que o script precisa esperar a página existir
- Usa o console e os breakpoints para descobrir por que um valor chegou errado

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.01`, `FE03.04`, `FE03.05`

FE03.C2 — Reagir a eventos da pessoa usuária

Registrar ouvintes de evento, entender que a função é chamada depois, pelo navegador, e tratar o envio de um formulário coletando e validando os dados.

Evidências observáveis

- Escreve o addEventListener que executa uma função ao clique
- Distingue passar a função como referência de executá-la na hora do registro
- Sabe por que o envio de um formulário precisa de preventDefault para não recarregar a página

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.01`, `FE03.02`, `FE03.04`

FE03.C3 — Criar, alterar e remover elementos e estilos no DOM

Construir elementos por código e anexá-los à página, alterar texto, imagem, atributos e classes, e renderizar na tela o resultado de um processamento de dados.

Evidências observáveis

- Dado um dado em memória, escreve o código que cria o elemento e o anexa ao contêiner certo
- Troca a classe de um elemento para mudar o estilo em vez de escrever estilo inline
- Prevê o que aparece na tela depois de um trecho que altera textContent e atributos

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.01`, `FE03.03`, `FE03.04`

FE03.C4 — Processar coleções com os métodos de array

Escolher entre forEach, map, filter, sort e reduce conforme o resultado desejado, e reconhecer quais deles devolvem um novo array e quais alteram o original.

Evidências observáveis

- Dado 'quero a lista só com os itens disponíveis', escolhe filter e não forEach
- Prevê o valor devolvido por um reduce que soma um campo da coleção
- Encadeia filter e map para transformar e selecionar na mesma passagem

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.03`, `FE03.07`

FE03.C5 — Lidar com código assíncrono

Entender que uma operação demorada não bloqueia o restante do programa, consumir uma promise, escrever funções com async/await e tratar o erro que vem de uma operação assíncrona.

Evidências observáveis

- Dado um trecho com setTimeout e código depois dele, prevê a ordem das saídas
- Reescreve um then/catch como async/await com try/catch
- Explica em uma frase por que o await precisa estar dentro de uma função async

Tipos de item: `multipla_escolha`, `analise_codigo`, `dissertativa` **Cursos de origem:** `FE03.02`, `FE03.04`

FE03.C6 — Consumir uma API com fetch e implementar o CRUD via HTTP

Fazer requisições ao servidor, escolher o método HTTP de cada operação do CRUD, interpretar a resposta e o status, e reconhecer o papel do cliente e do servidor na troca.

Evidências observáveis

- Associa cada operação do CRUD ao método HTTP correspondente
- Dado um fetch, aponta onde a resposta é convertida em objeto JavaScript
- Reconhece o significado dos status 200, 201, 404 e 500

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.06`, `FE03.07`

FE03.C7 — Persistir e validar dados no navegador

Guardar o estado da aplicação no localStorage entre sessões, trabalhar com datas, validar entradas com expressões regulares e usar Set para garantir valores únicos.

Evidências observáveis

- Sabe que o localStorage guarda texto e que o objeto precisa ser serializado para JSON
- Dada uma regra de preenchimento, escolhe a expressão regular que a valida
- Reconhece que o Set descarta valores repetidos ao ser construído

Tipos de item: `multipla_escolha`, `analise_codigo` **Cursos de origem:** `FE03.05`, `FE03.07`

FE03.C8 — Modularizar o código e usar o ambiente Node e NPM

Separar o código em módulos com import e export, e operar o ambiente de desenvolvimento: instalar pacotes com o NPM, distinguir dependência de desenvolvimento da de produção e rodar os scripts do projeto.

Evidências observáveis

- Escreve o export e o import que compartilham uma função entre dois arquivos
- Distingue uma dependência de desenvolvimento de uma de produção pelo que ela serve
- Explica o papel do package.json em um projeto front-end

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `FE03.01`, `FE03.08`

Cursos da trilha

FE03.01 — JavaScript: construindo páginas dinâmicas

8h · 48 atividades · atualizado em 2025-03-27 | Etapa: *Páginas dinâmicas e assíncronas*
https://www.alura.com.br/curso-online-construindo-paginas-dinamicas-javascript

Módulos da ementa. Integrando JavaScript ao projeto; Detectando interações; Construindo conteúdos dinâmicos; Modularizando o código; Criando novos arquivos.

Conceitos-chave. DOM, querySelector, addEventListener, createElement, append, checkbox, Date, estilo dinâmico, módulo, import, export default, parâmetro.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.01.O1	Ligar um script JavaScript à página e acessar seus elementos pelo DOM	aplicar	FE03.C1
FE03.01.O2	Detectar interações da pessoa usuária com ouvintes de evento	aplicar	FE03.C2
FE03.01.O3	Criar elementos HTML por código e anexá-los à página	aplicar	FE03.C3
FE03.01.O4	Construir conteúdo dinâmico aplicando estilos e formatando data e horário	aplicar	FE03.C3
FE03.01.O5	Separar o código em módulos e compartilhar funções com import e export	aplicar	FE03.C8

Competências alimentadas por este curso: FE03.C1 Integrar JavaScript à página, selecionar elementos e depurar no navegador; FE03.C2 Reagir a eventos da pessoa usuária; FE03.C3 Criar, alterar e remover elementos e estilos no DOM; FE03.C8 Modularizar o código e usar o ambiente Node e NPM.

Fora de escopo (o questionário não cobra). frameworks como React, Vue ou Angular; TypeScript; Web Components; service workers.

FE03.02 — JavaScript: entendendo promises e async/await

6h · 49 atividades · atualizado em 2025-03-25 | Etapa: *Páginas dinâmicas e assíncronas*
https://www.alura.com.br/curso-online-javascript-entendendo-promises-async-await

Módulos da ementa. Construindo a página; Assincronicidade; Manipulando tags; Coletando dados do formulário; Gerando interações com o formulário.

Conceitos-chave. assincronismo, Promise, then, catch, async, await, try/catch, setTimeout, Event Loop, Call Stack, Task Queue, FormData.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.02.O1	Explicar o que é código assíncrono e o papel do Event Loop, da Call Stack e da Task Queue	entender	FE03.C5
FE03.02.O2	Consumir o retorno de uma operação assíncrona com Promises	aplicar	FE03.C5
FE03.02.O3	Escrever funções assíncronas com async/await e tratar erros com try/catch	aplicar	FE03.C5

Objetivo	Enunciado	Bloom	Competência
FE03.02.04	Coletar e limitar os dados de um formulário antes de simular o envio	aplicar	FE03.C2

Competências alimentadas por este curso: FE03.C2 Reagir a eventos da pessoa usuária; FE03.C5 Lidar com código assíncrono.

Fora de escopo (o questionário não cobra). Web Workers; streams; generators assíncronos; AbortController.

FE03.03 — JavaScript: métodos de array

8h · 40 atividades · atualizado em 2024-05-15 | Etapa: *Páginas dinâmicas e assíncronas*
https://www.alura.com.br/curso-online-javascript-metodos-array

Módulos da ementa. forEach: função para cada elemento; map: novo array modificado; filter: aplicando filtros; sort: ordenando a lista; reduce: um único valor.

Conceitos-chave. array, forEach, map, filter, sort, reduce, callback, encadeamento de métodos, fetch.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.03.01	Percorrer uma coleção aplicando uma função a cada elemento com forEach	aplicar	FE03.C4
FE03.03.02	Transformar uma coleção com map e selecionar elementos com filter	aplicar	FE03.C4
FE03.03.03	Ordenar uma lista com sort e reduzir a coleção a um único valor com reduce	aplicar	FE03.C4
FE03.03.04	Renderizar na página o resultado do processamento feito pelos métodos de array	aplicar	FE03.C3

Competências alimentadas por este curso: FE03.C3 Criar, alterar e remover elementos e estilos no DOM; FE03.C4 Processar coleções com os métodos de array.

Fora de escopo (o questionário não cobra). imutabilidade e programação funcional avançada; estruturas de dados customizadas; performance e big O de coleções.

FE03.04 — JavaScript: manipulando elementos no DOM

6h · 47 atividades · atualizado em 2024-11-22 | Etapa: *Manipulação do DOM e as integrações com o backend*
https://www.alura.com.br/curso-online-javascript-manipulando-elementos-dom

Módulos da ementa. Iniciando o projeto Fokus; Manipulando elementos da página; Aplicando áudio e estilos; Desenvolvendo o temporizador; Finalizando o projeto.

Conceitos-chave. data attribute, classList, add, remove, toggle, textContent, src, Audio, setInterval, clearInterval, Date.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.04.O1	Selecionar elementos e ler ou alterar seus data attributes	aplicar	FE03.C1
FE03.04.O2	Alterar imagens, textos e classes de um elemento de forma dinâmica	aplicar	FE03.C3
FE03.04.O3	Criar eventos de interação que disparam as mudanças na interface	aplicar	FE03.C2
FE03.04.O4	Controlar um temporizador com setInterval e interrompê-lo com clearInterval	aplicar	FE03.C5

Competências alimentadas por este curso: FE03.C1 Integrar JavaScript à página, selecionar elementos e depurar no navegador; FE03.C2 Reagir a eventos da pessoa usuária; FE03.C3 Criar, alterar e remover elementos e estilos no DOM; FE03.C5 Lidar com código assíncrono.

Fora de escopo (o questionário não cobra). animações com Web Animations API; canvas; acessibilidade programática avançada; bibliotecas de manipulação do DOM.

FE03.05 — JavaScript: explorando a manipulação de elementos e da localStorage

8h · 53 atividades · atualizado em 2024-12-23 | Etapa: *Manipulação do DOM e as integrações com o backend*
https://www.alura.com.br/curso-online-javascript-manipulacao-elementos-localstorage

Módulos da ementa. Adicionando tarefas; Exibindo as tarefas no DOM; Editando tarefas; Integrando com o cronômetro; Removendo tarefas.

Conceitos-chave. localStorage, setItem, getItem, JSON.stringify, JSON.parse, estado da aplicação, array de objetos, depuração no navegador, breakpoint.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.05.O1	Gravar e recuperar dados no localStorage para persistir o estado entre sessões	aplicar	FE03.C7
FE03.05.O2	Gerenciar o estado de uma lista combinando array em memória e armazenamento local	analisar	FE03.C7
FE03.05.O3	Depurar o código no navegador para localizar a origem de um comportamento inesperado	analisar	FE03.C1

Competências alimentadas por este curso: FE03.C1 Integrar JavaScript à página, selecionar elementos e depurar no navegador; FE03.C7 Persistir e validar dados no navegador.

Fora de escopo (o questionário não cobra). IndexedDB; cookies e sessão no servidor; sincronização entre dispositivos; criptografia de dados locais.

FE03.06 — JavaScript: implementando CRUD com requisições HTTP

8h · 56 atividades · atualizado em 2026-01-06 | Etapa: *Manipulação do DOM e as integrações com o backend*
https://www.alura.com.br/curso-online-javascript-implementando-crud-requisicoes-http

Módulos da ementa. Primeira requisição; Requisições GET e POST; Requisição PUT; Requisição DELETE; Conhecendo o Axios.

Conceitos-chave. fetch, cliente-servidor, JSON Server, API, HTTP, GET, POST, PUT, DELETE,

`status code`, `CRUD`, `Axios`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.06.O1	Fazer uma requisição com fetch e exibir na interface os dados recebidos	aplicar	FE03.C6
FE03.06.O2	Implementar criar, ler, atualizar e deletar associando cada operação ao método HTTP correto	aplicar	FE03.C6
FE03.06.O3	Reconhecer o funcionamento do protocolo HTTP e o significado dos status de resposta	entender	FE03.C6
FE03.06.O4	Usar o Axios como alternativa ao fetch e inspecionar as requisições no navegador	aplicar	FE03.C6

Competências alimentadas por este curso: FE03.C6 Consumir uma API com fetch e implementar o CRUD via HTTP.

Fora de escopo (o questionário não cobra). autenticação e tokens; CORS em detalhe; escrever o back-end da API; GraphQL; WebSockets.

FE03.07 — JavaScript: evoluindo a sua aplicação com ES6+

8h · 61 atividades · atualizado em 2024-09-12 | Etapa: *Aprimorando e modernizando suas aplicações*
`https://www.alura.com.br/curso-online-javascript-evoluindo-sua-aplicacao-es6`

Módulos da ementa. Implementando um campo de busca; Favoritando um pensamento; Trabalhando com datas em JavaScript; Manipulando e exibindo datas; Implementando validações com expressões regulares; Evitando pensamentos duplicados.

Conceitos-chave. `ES6+`, `includes`, `filter`, `busca typeahead`, `PATCH`, `Date`, `UTC`, `formatação de data`, `expressão regular`, `Regex`, `Set`, `valores únicos`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.07.O1	Implementar uma busca typeahead combinando includes e filter sobre a coleção	aplicar	FE03.C4
FE03.07.O2	Atualizar parcialmente um recurso da API com o método PATCH	aplicar	FE03.C6
FE03.07.O3	Armazenar, converter e exibir datas corretamente com o objeto Date	aplicar	FE03.C7
FE03.07.O4	Validar o conteúdo de um campo com expressões regulares	aplicar	FE03.C7
FE03.07.O5	Usar Set para garantir que uma coleção não tenha valores duplicados	aplicar	FE03.C7

Competências alimentadas por este curso: FE03.C4 Processar coleções com os métodos de array; FE03.C6 Consumir uma API com fetch e implementar o CRUD via HTTP; FE03.C7 Persistir e validar dados no navegador.

Fora de escopo (o questionário não cobra). Proxy e Reflect; decorators; optional chaining em cadeias profundas; internacionalização completa com Intl.

FE03.08 — Node.js e terminal: dominando o ambiente de desenvolvimento front-end

8h · 61 atividades · atualizado em 2025-02-12 | Etapa: *Aprimorando e modernizando suas aplicações*
https://www.alura.com.br/curso-online-node-js-terminal-ambiente-desenvolvimento-front-end

Módulos da ementa. Conhecendo o Node.js e o terminal; Gerenciando pacotes com NPM; Explorando scripts, pacotes locais e globais; Usando uma biblioteca externa; Deploy do projeto.

Conceitos-chave. `Node.js`, `terminal`, `NPM`, `package.json`, `dependência de desenvolvimento`, `dependência de produção`, `versionamento semântico`, `script NPM`, `ESLint`, `Prettier`, `CDN`, `Vite`, `deploy`.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
FE03.08.O1	Configurar um ambiente front-end com Node e NPM, distinguindo dependências de desenvolvimento das de produção	entender	FE03.C8
FE03.08.O2	Instalar pacotes, criar scripts no package.json e usar ESLint, Prettier e Vite no projeto	aplicar	FE03.C8

Competências alimentadas por este curso: FE03.C8 Modularizar o código e usar o ambiente Node e NPM.

Fora de escopo (o questionário não cobra). Node.js no servidor; monorepos; configuração avançada de bundler; Docker; pipelines de CI/CD.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Lê HTML e CSS, mas não prevê o que um trecho de JavaScript faz com a página nem seleciona um elemento corretamente.
basico	50–69	Registra um clique e altera um texto na tela; erra entre os métodos de array, esquece preventDefault e trata promise como se fosse o valor.
adequado	70–84	Manipula o DOM criando e alterando elementos, escolhe o método de array adequado, escreve função async/await e consome uma API com fetch implementando o CRUD.
avancado	85–100	Prevê a ordem de execução de código assíncrono, diagnostica no navegador por que um valor chegou errado, persiste estado com localStorage e opera o ambiente Node/NPM do projeto.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. A trilha pressupõe FE01 concluída: os itens não devem cobrar HTML e CSS por si, apenas como suporte ao JavaScript. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 45 minutos estimados. Por tipo na matriz: multipla_escolha 10 · analise_codigo 8 · dissertativa 2. Por nível cognitivo: lembrar/entender 5 · aplicar 11 · analisar/avaliar 4.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com código	Foco da linha
C1 JavaScript na página e seletores	2	Múltipla escolha	1	seletor que recupera o elemento, script rodando antes da página
C2 Eventos	2	Múltipla escolha	1	referência vs chamada da função, preventDefault no submit
C3 Criar e alterar elementos no DOM	3	Múltipla escolha	2	createElement e append, classe vs style, textContent
C4 Métodos de array	2	Múltipla escolha	2	qual método para o resultado desejado, saída de um reduce
C4 Métodos de array	1	Múltipla resposta	0	quais métodos devolvem um novo array sem alterar o original
C5 Código assíncrono	2	Múltipla escolha	1	ordem das saídas com setTimeout, then/catch vs async/await
C5 Código assíncrono	1	Dissertativa	0	por que o valor chegou como promise pendente e como corrigir
C6 fetch e CRUD por HTTP	2	Múltipla escolha	1	método HTTP de cada operação, onde entra o .json()
C6 fetch e CRUD por HTTP	1	Múltipla resposta	0	quais passos uma requisição de criação precisa cumprir
C7 localStorage, datas, regex e Set	2	Múltipla escolha	0	stringify e parse, mês do Date, Set sem duplicados
C8 Módulos, Node e NPM	1	Múltipla escolha	0	export default vs nomeado, papel do package.json
C8 Módulos, Node e NPM	1	Dissertativa	0	diferença entre dependência de desenvolvimento e de produção

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 8 com código no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/FE03-front-end-javascript.json‘ (‘multipla_escolha‘ 10 + ‘analise_codigo‘ 8) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta, e a coluna "Com código" reproduz os 8 de ‘analise_codigo‘ — a plataforma não tem botão para esse tipo.

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - FRONT-END COM JAVASCRIPT

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. O estudante já constrói
```

```
páginas com HTML e CSS e agora aprendeu a torná-las dinâmicas com JavaScript. Os projetos
fio condutor são aplicações interativas: lista de tarefas, temporizador Pomodoro, mural de
pensamentos com CRUD em API e catálogo de livros com filtros.

ASSUNTOS

C1 - JavaScript na página, seletores e depuração
  Cobrar: dado um elemento no HTML, escrever o seletor que o recupera em JavaScript;
  explicar o que é o DOM e por que o script precisa esperar a página existir; usar o console
  e os breakpoints para descobrir por que um valor chegou errado.
  Erros típicos, para virar alternativa errada: rodar o script antes de a página carregar e
  receber null no seletor; confundir querySelector, que devolve um elemento, com
  querySelectorAll, que devolve uma lista; achar que alterar o DOM muda o arquivo HTML no
  disco; depurar só com console.log, sem olhar a mensagem de erro que o navegador já deu.

C2 - Eventos
  Cobrar: escrever o addEventListener que executa uma função ao clique; distinguir passar a
  função como referência de executá-la na hora do registro; saber por que o envio de um
  formulário precisa de preventDefault.
  Erros típicos: escrever addEventListener('click', minhaFuncao()) e executar a função no
  registro; esquecer preventDefault no submit e ver a página recarregar perdendo os dados;
  registrar o ouvinte em um elemento que ainda não existe na página; registrar o mesmo
  ouvinte várias vezes e a ação acontecer em duplicidade.

C3 - Criar, alterar e remover elementos no DOM
  Cobrar: dado um dado em memória, escrever o código que cria o elemento e o anexa ao
  contêiner certo; trocar a classe de um elemento para mudar o estilo em vez de escrever
  estilo inline; prever o que aparece na tela depois de um trecho que altera textContent e
  atributos.
  Erros típicos: criar o elemento com createElement e esquecer de anexá-lo com append; usar
  innerHTML com conteúdo vindo da pessoa usuária em vez de textContent; alterar style no
  script onde uma classe CSS resolveria; confundir textContent com value em campos de
  formulário.

C4 - Métodos de array
  Cobrar: dado "quero a lista só com os itens disponíveis", escolher filter e não forEach;
  prever o valor devolvido por um reduce que soma um campo da coleção; encadear filter e map
  para transformar e selecionar na mesma passagem.
  Erros típicos: usar forEach esperando que ele devolva um novo array; esquecer o return
  dentro do callback de map ou filter e receber undefined; achar que map filtra elementos
  quando o callback devolve false; usar sort sem função de comparação e ordenar números como
  texto.

C5 - Código assíncrono
  Cobrar: dado um trecho com setTimeout e código depois dele, prever a ordem das saídas;
  reescrever um then/catch como async/await com try/catch; explicar em uma frase por que o
  await precisa estar dentro de uma função async.
  Erros típicos: usar o resultado de uma função assíncrona antes de ela terminar e receber
  uma promise pendente; esquecer o await e tratar a promise como se fosse o valor; achar que
  o setTimeout de 0 ms executa imediatamente, antes do código seguinte; esquecer o
  clearInterval e deixar o temporizador rodando para sempre.

C6 - fetch e CRUD por HTTP
  Cobrar: associar cada operação do CRUD ao método HTTP correspondente; dado um fetch,
  apontar onde a resposta é convertida em objeto JavaScript; reconhecer o significado dos
  status 200, 201, 404 e 500.
  Erros típicos: usar GET para enviar dados que deveriam ir no corpo de um POST; esquecer o
  .json() e tratar a resposta como se já fosse o objeto; confundir PUT, que substitui o
  recurso, com PATCH, que altera parte dele; achar que o fetch devolve os dados direto, sem
  passar pela promise.
```

```
C7 - localStorage, datas, expressões regulares e Set
  Cobrar: saber que o localStorage guarda texto e que o objeto precisa ser serializado para
  JSON; dada uma regra de preenchimento, escolher a expressão regular que a valida;
  reconhecer que o Set descarta valores repetidos ao ser construído.
  Erros típicos: gravar um objeto no localStorage sem JSON.stringify e salvar o texto
  [object Object]; ler do localStorage e esquecer o JSON.parse antes de usar como objeto;
  achar que o localStorage é compartilhado entre navegadores ou dispositivos; confundir o
  mês do objeto Date, que começa em zero, com o mês do calendário.

C8 - Módulos, Node e NPM
  Cobrar: escrever o export e o import que compartilham uma função entre dois arquivos;
  distinguir uma dependência de desenvolvimento de uma de produção pelo que ela serve;
  explicar o papel do package.json em um projeto front-end.
  Erros típicos: confundir export default com export nomeado na hora de importar; esquecer o
  type="module" no script e o import não funcionar no navegador; instalar como dependência
  de produção uma ferramenta que só serve para desenvolver; versionar a pasta node_modules
  junto com o projeto.

FORA DE ESCOPO - não cobre nada disso: React, Vue, Angular e qualquer framework de
interface, TypeScript, Web Components, service workers, Web Workers, streams, canvas,
IndexedDB, cookies e sessão no servidor, autenticação e tokens, CORS em detalhe, escrever o
back-end da API, GraphQL, WebSockets, Node.js no servidor, monorepos, Docker e pipelines de
CI/CD. Também não cobre HTML e CSS por si: eles aparecem só como suporte ao JavaScript.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: lista de tarefas, catálogo de jogos,
  temporizador de estudo, playlist, mural de recados, placar de um campeonato.
- Quando o item trouxer código, use JavaScript com no máximo 15 linhas, sem framework e sem
  biblioteca externa - a única exceção é o Axios, nos itens de C6. Nos itens de código,
  descreva em texto o efeito esperado na página; não dependa de o estudante ver a tela.
- Nada de pegadinha de sintaxe fora do que a trilha ensinou. Se o erro a ser detectado for
  de digitação e não de conceito, o item não serve.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie conhecimento de JavaScript, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 5 itens de compreensão, 11 de aplicação e 4
  de análise ou avaliação.
```

UX02 — Game Design

Escola (ITT)	UX & Design
Nome na planilha ITT	Game Design
Nome atual na Alura	Game Design
Página da formação	`https://www.alura.com.br/formacao-game-design`
Carga horária	32h em 4 cursos
Pré-requisitos	nenhum
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, sem experiência prévia em projeto de jogos; a trilha é de concepção e documentação, não de programação
Projeto fio condutor	Um jogo próprio levado da ideia ao pitch: Game Design Document, protótipo, Game Design Canvas e MVP, direção de arte e apresentação comercial
Matriz	`cursos/UX02-game-design.json`
Prompt	`prompts/UX02.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`UX02.C1`	Analisar o design de jogos publicados e avaliar a maturidade do próprio projeto	0.09	2	analisar
`UX02.C2`	Definir o conceito do jogo	0.14	3	aplicar
`UX02.C3`	Documentar o projeto em GDD e em Game Design Canvas	0.14	3	aplicar
`UX02.C4`	Projetar a mecânica: loop central, levels e protótipo	0.14	3	analisar
`UX02.C5`	Equilibrar o sistema de recompensas e punições	0.10	2	avaliar
`UX02.C6`	Construir narrativa e personagens por arquétipos	0.10	2	aplicar
`UX02.C7`	Dirigir os aspectos visuais do jogo	0.10	2	aplicar
`UX02.C8`	Avaliar a viabilidade comercial e apresentar o jogo	0.19	3	avaliar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

UX02.C1 — Analisar o design de jogos publicados e avaliar a maturidade do próprio projeto

Olhar um jogo já lançado e identificar as escolhas de design por trás dele, e aplicar o mesmo olhar crítico ao próprio jogo por autoavaliação e playtest.

Evidências observáveis

- Dado um jogo conhecido, aponta qual escolha de design produz uma sensação específica em quem joga
- Distingue a opinião pessoal sobre o jogo da análise da escolha de design
- Explica o que um playtest revela que a autoavaliação não revela

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX02.01`, `UX02.04`

UX02.C2 — Definir o conceito do jogo

Responder o que o jogo é, do que ele trata e o que a pessoa faz nele, escolhendo o tema, as ações centrais e as recompensas que sustentam a experiência.

Evidências observáveis

- Dada uma ideia vaga de jogo, formula a resposta das três perguntas que a delimitam
- Distingue o tema do jogo da mecânica do jogo
- Escolhe a ação central coerente com a experiência que o jogo quer provocar

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX02.01`

UX02.C3 — Documentar o projeto em GDD e em Game Design Canvas

Registrar as decisões do jogo em um Game Design Document, sintetizá-las em um Game Design Canvas e delimitar o MVP — o mínimo que já entrega a experiência.

Evidências observáveis

- Dado um conjunto de decisões, aponta em que seção do GDD cada uma entra
- Distingue o que o GDD detalha do que o Canvas resume
- Separa, em uma lista de funcionalidades, o que entra no MVP do que fica para depois

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX02.01`, `UX02.02`

UX02.C4 — Projetar a mecânica: loop central, levels e protótipo

Identificar o loop central de ações que o jogo repete, desdobrá-lo em levels com dificuldade progressiva e modularidade, e testar a ideia em um protótipo.

Evidências observáveis

- Dado um jogo, descreve o ciclo de ações que a pessoa repete do começo ao fim
- Explica o que a modularidade de um level resolve na produção
- Reconhece que o protótipo serve para testar a mecânica, não para parecer bonito

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX02.01`, `UX02.02`

UX02.C5 — Equilibrar o sistema de recompensas e punições

Estruturar o que o jogo dá e o que o jogo cobra de modo que a pessoa continue jogando, reconhecendo quando a punição desestimula e quando a recompensa perde valor.

Evidências observáveis

- Dado um sistema de punição, avalia se ele ensina ou apenas frustra
- Reconhece que recompensa dada o tempo todo deixa de recompensar
- Relaciona a recompensa à ação que o jogo quer incentivar

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX02.02`

UX02.C6 — Construir narrativa e personagens por arquétipos

Criar a story line do jogo e definir heróis, vilões e coadjuvantes a partir de arquétipos, articulando narrativa, personagem e cenário.

Evidências observáveis

- Dado um arquétipo, identifica que papel narrativo o personagem cumpre no jogo
- Distingue a story line do roteiro de cada cena
- Relaciona a escolha do cenário à narrativa que o jogo conta

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX02.01`, `UX02.02`, `UX02.03`

UX02.C7 — Dirigir os aspectos visuais do jogo

Aplicar direção de arte ao projeto: concept art, psicologia das cores e acordes cromáticos, contraste e paralaxe no cenário, e o fluxo das interfaces.

Evidências observáveis

- Dada a sensação desejada em uma cena, escolhe a paleta coerente com ela
- Explica para que serve o contraste na leitura visual de um cenário
- Reconhece o papel do mood board e do concept art antes da produção da arte

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX02.03`

UX02.C8 — Avaliar a viabilidade comercial e apresentar o jogo

Identificar público-alvo e oportunidade no mercado de games, comparar formas de monetização adequadas ao jogo, dimensionar a produção e montar um pitch de apresentação.

Evidências observáveis

- Dado um jogo, escolhe a forma de monetização coerente com ele e justifica
- Identifica o público-alvo a partir das características do jogo, não do desejo de alcançar todos
- Nomeia o que um pitch precisa responder: o que é o jogo, para quem, e o que se busca

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX02.03`, `UX02.04`

Cursos da trilha

UX02.01 — Game Design: definindo os princípios de um jogo digital

8h · 47 atividades · atualizado em 2025-05-14 | Etapa: *Entendendo como nasce um game*
https://www.alura.com.br/curso-online-game-design-definindo-principios-jogo-digital

Módulos da ementa. Entendendo o Game Design; Definindo o que é o seu jogo; Esboçando um protótipo; Pensando o Narrative Design; Pensando imagens e sons do jogo.

Conceitos-chave. game designer, referência, três perguntas de Jared Sorensen, tema, ação, recompensa, protótipo, prototipagem, interatividade, storytelling, jornada do herói, story line, personagem, cenário, sound design, GDD.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX02.01.O1	Analisar referências de jogos publicados e identificar a lógica de criação por trás delas	analisar	UX02.C1
UX02.01.O2	Delimitar o jogo respondendo às três perguntas de Jared Sorensen e definindo o tema	aplicar	UX02.C2
UX02.01.O3	Escolher as ações e as recompensas centrais do jogo de forma coerente entre si	aplicar	UX02.C2
UX02.01.O4	Esquematizar um protótipo do jogo e agregar interatividade a ele	aplicar	UX02.C4
UX02.01.O5	Criar a story line, os personagens e os cenários do jogo	aplicar	UX02.C6
UX02.01.O6	Reunir referências visuais e de sound design para o projeto	aplicar	UX02.C7
UX02.01.O7	Registrar as decisões do projeto em um Game Design Document completo	aplicar	UX02.C3

Competências alimentadas por este curso: UX02.C1 Analisar o design de jogos publicados e avaliar a maturidade do próprio projeto; UX02.C2 Definir o conceito do jogo; UX02.C3 Documentar o projeto em GDD e em Game Design Canvas; UX02.C4 Projetar a mecânica: loop central, levels e protótipo; UX02.C6 Construir narrativa e personagens por arquétipos; UX02.C7 Dirigir os aspectos visuais do jogo.

Fora de escopo (o questionário não cobra). programação do jogo; uso de engines; produção de arte final; composição musical; publicação em lojas.

UX02.02 — Game Design: viabilizando o jogo como produto

8h · 46 atividades · atualizado em 2024-01-12 | Etapa: *Transformando o game em produto*
https://www.alura.com.br/curso-online-game-design-viabilizando-jogo-produto

Módulos da ementa. Conhecendo o GDC; Aplicando o Design Thinking; Definindo Levels; Refinando recompensas e punições; Aprimorando os personagens.

Conceitos-chave. Game Design Canvas, MVP, Design Thinking, loop central, level design, modularidade, recompensa, punição, arquétipo, herói, vilão.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX02.02.O1	Traduzir o GDD em um Game Design Canvas que dê visão clara do projeto	aplicar	UX02.C3

Objetivo	Enunciado	Bloom	Competência
UX02.02.02	Delimitar o MVP do jogo separando o essencial do que fica para depois	analisar	UX02.C3
UX02.02.03	Aplicar Design Thinking para explorar o problema de jogabilidade e idealizar soluções	aplicar	UX02.C4
UX02.02.04	Identificar o loop central do jogo e definir os levels com modularidade	analisar	UX02.C4
UX02.02.05	Estruturar o sistema de recompensas e punições para engajar quem joga	avaliar	UX02.C5
UX02.02.06	Construir heróis e vilões a partir de arquétipos narrativos	aplicar	UX02.C6

Competências alimentadas por este curso: UX02.C3 Documentar o projeto em GDD e em Game Design Canvas; UX02.C4 Projetar a mecânica: loop central, levels e protótipo; UX02.C5 Equilibrar o sistema de recompensas e punições; UX02.C6 Construir narrativa e personagens por arquétipos.

Fora de escopo (o questionário não cobra). balanceamento por telemetria; economia de jogo com moeda virtual; produção e cronograma detalhado; contratação de equipe.

UX02.03 — Game design: refinando aspectos visuais e artísticos do jogo

8h · 41 atividades · atualizado em 2024-01-29 | Etapa: *Viabilizando o game*
https://www.alura.com.br/curso-online-game-design-refinando-aspectos-visuais-artisticos-jogo

Módulos da ementa. Dimensionando o projeto; Entendendo direção de arte; Definindo personagens; Definindo cenários; Definindo interfaces.

Conceitos-chave. pipeline, backlog, direção de arte, concept art, psicologia das cores, acorde cromático, mood board, arquétipo visual, contraste, linha de ação, paralaxe, fluxo de interfaces, UI.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX02.03.01	Dimensionar a pipeline de produção de arte e montar o backlog do projeto	avaliar	UX02.C8
UX02.03.02	Aplicar psicologia das cores e acordes cromáticos na direção de arte do jogo	aplicar	UX02.C7
UX02.03.03	Conceituar o visual de protagonista, coadjuvantes e vilãs a partir de arquétipos visuais	aplicar	UX02.C6
UX02.03.04	Definir cenários usando contraste, linhas de ação e paralaxe	aplicar	UX02.C7
UX02.03.05	Definir o fluxo entre as interfaces do jogo a partir de referências de UI	aplicar	UX02.C7

Competências alimentadas por este curso: UX02.C6 Construir narrativa e personagens por arquétipos; UX02.C7 Dirigir os aspectos visuais do jogo; UX02.C8 Avaliar a viabilidade comercial e apresentar o jogo.

Fora de escopo (o questionário não cobra). produção da arte em software gráfico; modelagem 3D; animação; shaders e efeitos técnicos.

UX02.04 — Game Design: entenda estratégias para viabilizar e monetizar o seu jogo

8h · 45 atividades · atualizado em 2024-05-16 | Etapa: *Viabilizando o game*
https://www.alura.com.br/curso-online-game-design-estrategias-viabilizar-monetizar-seu-jogo

Módulos da ementa. Meu jogo está pronto?; Um jogo, um produto; Meios e estratégias de negócio; Finanças e continuidade; Apresentando seu pitch.

Conceitos-chave. autoavaliação, playtest, mercado de games, público-alvo, formação de preço, monetização, jogo como serviço, jogo como plataforma, propriedade intelectual, plano financeiro, pitch.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX02.04.O1	Avaliar a maturidade do próprio jogo por autoavaliação e playtest	avaliar	UX02.C1
UX02.04.O2	Identificar o público-alvo do jogo e as oportunidades de inserção no mercado de games	analisar	UX02.C8
UX02.04.O3	Comparar formas de monetização e escolher a adequada ao jogo projetado	avaliar	UX02.C8
UX02.04.O4	Montar um plano financeiro e reconhecer o papel da propriedade intelectual	aplicar	UX02.C8
UX02.04.O5	Construir um pitch que apresente o jogo, o mercado e o que se busca	aplicar	UX02.C8

Competências alimentadas por este curso: UX02.C1 Analisar o design de jogos publicados e avaliar a maturidade do próprio projeto; UX02.C8 Avaliar a viabilidade comercial e apresentar o jogo.

Fora de escopo (o questionário não cobra). contabilidade e abertura de empresa; negociação de contratos; campanhas de marketing pagas; captação formal de investimento.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Fala sobre jogos a partir do gosto pessoal; não distingue tema de mecânica nem sabe o que um GDD registra.
basico	50–69	Descreve a ideia do próprio jogo e reconhece os nomes dos artefatos; erra ao delimitar o MVP e ao identificar o loop central.
adequado	70–84	Delimita o conceito do jogo, documenta as decisões em GDD e Canvas, identifica o loop central, define levels e constrói personagens por arquétipos.
avancado	85–100	Avalia se um sistema de recompensa e punição sustenta o engajamento, justifica a direção de arte pela experiência pretendida e defende uma estratégia de monetização e um pitch coerentes com o jogo.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Atenção: a Alura declara os cursos 2, 3 e 4 para profissionais da área com projeto em andamento, e o curso 4 trata de viabilidade comercial e captação. A matriz cobra o raciocínio de design e de produto, não a prática profissional de mercado. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 40 minutos estimados. Por tipo na matriz: multipla_escolha 18 · dissertativa 2. Por nível cognitivo: lembrar/entender 6 · aplicar 9 · analisar/avaliar 5.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com estímulo	Foco da linha
C1 Analisar jogos e maturidade do projeto	2	Múltipla escolha	1	qual escolha de design produz a sensação, o que o playtest revela
C2 Conceito do jogo	3	Múltipla escolha	1	tema vs mecânica, ação central, recompensa ligada à ação
C3 GDD, Canvas e MVP	2	Múltipla escolha	1	em que seção do GDD a decisão entra, GDD vs Canvas
C3 GDD, Canvas e MVP	1	Múltipla resposta	0	quais funcionalidades entram no MVP
C4 Loop central, levels e protótipo	2	Múltipla escolha	1	qual é o loop central, para que serve o protótipo
C4 Loop central, levels e protótipo	1	Dissertativa	0	descrever o loop central de um jogo e por que ele sustenta a experiência
C5 Recompensas e punições	2	Múltipla escolha	1	punição que ensina vs que frustra, recompensa que perde valor
C6 Narrativa e arquétipos	2	Múltipla escolha	0	papel narrativo do arquétipo, story line vs roteiro de cena
C7 Direção de arte e interfaces	2	Múltipla escolha	1	paleta coerente com o clima, contraste na leitura do cenário
C8 Viabilidade e pitch	1	Múltipla escolha	0	público-alvo a partir do jogo, o que o pitch precisa responder
C8 Viabilidade e pitch	1	Múltipla resposta	0	quais formas de monetização cabem no jogo descrito
C8 Viabilidade e pitch	1	Dissertativa	0	escolher uma monetização para o próprio jogo e justificar

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 6 com estímulo no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/UX02-game-design.json‘ (‘multipla_escolha‘ 18) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta. A matriz não usa ‘analise_codigo‘ — a trilha não tem programação —, por isso a coluna aqui é "Com estímulo" e não "Com código".

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só.

Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - GAME DESIGN

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. A trilha é de concepção e
documentação de jogos, não de programação: o estudante levou um jogo próprio da ideia ao
pitch, passando por Game Design Document, protótipo, Game Design Canvas e MVP, direção de
arte e apresentação comercial.

ASSUNTOS

C1 - Analisar jogos publicados e a maturidade do próprio projeto
  Cobrar: dado um jogo conhecido, apontar qual escolha de design produz uma sensação
  específica em quem joga; distinguir a opinião pessoal sobre o jogo da análise da escolha
  de design; explicar o que um playtest revela que a autoavaliação não revela.
  Erros típicos, para virar alternativa errada: confundir analisar o design com dizer se
  gostou do jogo; analisar só o gráfico e ignorar a mecânica; fazer playtest apenas com quem
  já conhece o projeto; tratar a referência como algo a copiar em vez de algo a entender.

C2 - Conceito do jogo
  Cobrar: dada uma ideia vaga de jogo, formular a resposta das três perguntas que a
  delimitam; distinguir o tema do jogo da mecânica do jogo; escolher a ação central coerente
  com a experiência que o jogo quer provocar.
  Erros típicos: descrever a história e achar que isso define o jogo; listar várias ações
  centrais e não escolher nenhuma; confundir o gênero do jogo com o tema; definir recompensa
  só como pontuação, sem relação com a ação central.

C3 - GDD, Game Design Canvas e MVP
  Cobrar: dado um conjunto de decisões, apontar em que seção do GDD cada uma entra;
  distinguir o que o GDD detalha do que o Canvas resume; separar, em uma lista de
  funcionalidades, o que entra no MVP do que fica para depois.
  Erros típicos: tratar o GDD como documento escrito uma vez e nunca revisado; confundir MVP
  com versão incompleta e sem graça do jogo; colocar no MVP tudo que foi idealizado; usar o
  Canvas como substituto do GDD em vez de síntese dele.

C4 - Loop central, levels e protótipo
  Cobrar: dado um jogo, descrever o ciclo de ações que a pessoa repete do começo ao fim;
  explicar o que a modularidade de um level resolve na produção; reconhecer que o protótipo
  serve para testar a mecânica, não para parecer bonito.
  Erros típicos: confundir o loop central com a sequência da história; projetar levels por
  quantidade em vez de por progressão de desafio; fazer um protótipo com arte final antes de
  validar a mecânica; achar que dificuldade maior é só colocar mais inimigos.

C5 - Recompensas e punições
  Cobrar: dado um sistema de punição, avaliar se ele ensina ou apenas frustra; reconhecer
  que recompensa dada o tempo todo deixa de recompensar; relacionar a recompensa à ação que
  o jogo quer incentivar.
  Erros típicos: punir o erro com perda de progresso longo e afastar quem está aprendendo;
  distribuir recompensas sem relação com o esforço; confundir dificuldade com punição; achar
  que mais recompensa sempre significa mais engajamento.

C6 - Narrativa e arquétipos
  Cobrar: dado um arquétipo, identificar que papel narrativo o personagem cumpre no jogo;
  distinguir a story line do roteiro de cada cena; relacionar a escolha do cenário à
  narrativa que o jogo conta.
  Erros típicos: tratar arquétipo como estereótipo fixo e sem variação; criar personagem
  detalhado que não cumpre nenhuma função no jogo; confundir a jornada do herói com a ordem
  obrigatória dos levels; escrever narrativa longa que a mecânica do jogo não sustenta.
```

```
C7 - Direção de arte e interfaces
  Cobrar: dada a sensação desejada em uma cena, escolher a paleta coerente com ela; explicar
  para que serve o contraste na leitura visual de um cenário; reconhecer o papel do mood
  board e do concept art antes da produção da arte.
  Erros típicos: escolher cores por gosto pessoal, sem relação com o clima da cena; deixar
  personagem e fundo com o mesmo valor de contraste e perder a leitura; confundir concept art
  com a arte final do jogo; projetar as telas de interface sem pensar no fluxo entre elas.

C8 - Viabilidade e pitch
  Cobrar: dado um jogo, escolher a forma de monetização coerente com ele e justificar;
  identificar o público-alvo a partir das características do jogo, não do desejo de alcançar
  todos; nomear o que um pitch precisa responder: o que é o jogo, para quem, e o que se
  busca.
  Erros típicos: definir o público-alvo como "todo mundo que gosta de jogos"; escolher a
  monetização pelo que dá mais dinheiro, ignorando o tipo de jogo; montar pitch que só conta
  a história e não diz o que se pede ao investidor; confundir propriedade intelectual com o
  arquivo do jogo; dimensionar a produção sem considerar o tamanho do time.

FORA DE ESCOPO - não cobre nada disso: programação do jogo, uso de engines, produção de
arte final em software gráfico, modelagem 3D, animação, shaders, composição musical,
publicação em lojas, balanceamento por telemetria, economia de jogo com moeda virtual,
cronograma detalhado de produção, contratação de equipe, contabilidade e abertura de
empresa, negociação de contratos, campanhas de marketing pagas e captação formal de
investimento.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: jogo de plataforma, jogo de puzzle, jogo de
  terror leve, jogo de corrida, jogo cooperativo de escola, jogo de cartas digital.
- Quando o item trouxer estímulo, use um trecho curto de GDD, uma cena de jogo descrita em
  texto, uma lista de funcionalidades ou uma paleta descrita por nome de cor - no máximo 8
  linhas. Nunca dependa de o estudante ver uma imagem.
- Não cobre nome de ferramenta, modelo de template ou ordem de preenchimento de documento:
  avalie a decisão de design, não a memória do formulário.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie raciocínio de game design, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 6 itens de compreensão, 9 de aplicação e 5
  de análise ou avaliação.
```

UX03 — Blender

Escola (ITT)	UX & Design
Nome na planilha ITT	Blender
Nome atual na Alura	Blender
Página da formação	`https://www.alura.com.br/formacao-blender`
Carga horária	40h em 5 cursos
Pré-requisitos	nenhum
Público-alvo	Adolescentes do ensino fundamental II e médio atendidos pela ITT, sem experiência prévia em modelagem 3D; a trilha vai da interface do Blender à animação de um personagem
Projeto fio condutor	Peças 3D construídas do zero: um ambiente minimalista blocado, uma espada lowpoly modelada e pintada à mão, um personagem modelado poly by poly e, por fim, riggado e animado
Matriz	`cursos/UX03-blender.json`
Prompt	`prompts/UX03.md`

Competências avaliadas

Código	Competência	Peso	Itens	Bloom
`UX03.C1`	Operar a interface do Blender e navegar no espaço 3D	0.09	2	entender
`UX03.C2`	Modelar por blocagem e com Box Modeling	0.15	3	aplicar
`UX03.C3`	Controlar o fluxo de malha e modelar Poly by Poly	0.15	3	analisar
`UX03.C4`	Usar modificadores e propriedades de objeto	0.10	2	aplicar
`UX03.C5`	Abrir e organizar o mapa UV	0.12	2	aplicar
`UX03.C6`	Texturizar e pintar com a técnica Hand Paint	0.14	3	aplicar
`UX03.C7`	Montar o rig do personagem	0.15	3	aplicar
`UX03.C8`	Animar com keyframes e renderizar o resultado	0.10	2	aplicar
Total		**1.00**	**20**	

Detalhe de cada competência: o que ela significa e as evidências que a demonstram. Os erros comuns que viram distratores não se repetem aqui porque aparecem na íntegra no texto colável do prompt, adiante.

`UX03.C1` — **Operar a interface do Blender e navegar no espaço 3D**

Reconhecer os painéis e os modos de trabalho, navegar no viewport, organizar a área de trabalho com imagem de referência e usar atalhos para agilizar a produção.

Evidências observáveis

- Dada uma tarefa, aponta em qual modo de trabalho ela se realiza (objeto ou edição)
- Reconhece o papel da imagem de referência e do modelsheet antes de começar a modelar
- Distingue mover a câmera do viewport de mover o objeto na cena

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.01`, `UX03.02`

`UX03.C2` — **Modelar por blocagem e com Box Modeling**

Construir a forma partindo de primitivas simples e desenvolvê-la com extrusão e cortes, do ambiente blocado ao objeto lowpoly acabado.

Evidências observáveis

- Dada uma forma a construir, escolhe a primitiva de partida adequada
- Explica o que a extrusão faz com a face selecionada
- Reconhece para que serve a blocagem antes do detalhamento

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.01`, `UX03.02`

`UX03.C3` — **Controlar o fluxo de malha e modelar Poly by Poly**

Comparar as técnicas de modelagem, construir a malha polígono a polígono acompanhando a forma do personagem, e reconhecer o papel do fluxo de malha e da retopologia.

Evidências observáveis

- Dada uma peça a modelar, justifica a escolha entre Box Modeling e Poly by Poly
- Explica por que o fluxo de malha importa em um modelo que vai ser animado
- Diz em uma frase o que a retopologia resolve

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX03.02`, `UX03.04`

`UX03.C4` — **Usar modificadores e propriedades de objeto**

Aplicar modificadores para obter resultados sem alterar a malha base e ajustar as propriedades do objeto conforme o efeito desejado.

Evidências observáveis

- Dado um resultado desejado, escolhe o modificador que o produz
- Reconhece que o modificador é não destrutivo até ser aplicado
- Explica a vantagem de modelar metade e espelhar com modificador

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.01`, `UX03.04`

UX03.C5 — Abrir e organizar o mapa UV

Entender o mapa UV como a planificação da malha, abri-lo com unwrap, testar a distribuição e escolher a resolução adequada da textura.

Evidências observáveis

- Explica em uma frase o que o mapa UV representa
- Reconhece um UV mal distribuído pelo esticamento da textura no modelo
- Relaciona a resolução do mapa ao tamanho do objeto na cena

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.03`

UX03.C6 — Texturizar e pintar com a técnica Hand Paint

Organizar os materiais do modelo, pintar a textura à mão sobre o mapa UV e preparar a peça colorida para uso em uma engine de jogo.

Evidências observáveis

- Distingue o material do modelo da textura aplicada a ele
- Reconhece o que a pintura Hand Paint resolve que a cor lisa não resolve
- Sabe o que precisa estar pronto para levar o prop a uma engine

Tipos de item: `multipla_escolha`, `dissertativa` **Cursos de origem:** `UX03.03`, `UX03.04`

UX03.C7 — Montar o rig do personagem

Construir o esqueleto com bones, criar IK e controladores para facilitar a animação, e vincular a malha ao esqueleto ajustando a pintura de peso.

Evidências observáveis

- Dada uma deformação errada no braço, aponta que a pintura de peso é a causa
- Explica o que o controlador resolve no fluxo de trabalho de animação
- Reconhece a função da IK em um membro do personagem

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.05`

UX03.C8 — Animar com keyframes e renderizar o resultado

Criar a animação por keyframes, organizar trechos com a NonLinear Animation, iluminar a cena, escolher o motor de render e exportar o resultado final.

Evidências observáveis

- Explica o que um keyframe marca e o que o Blender faz entre dois deles
- Reconhece que a escolha do motor de render muda tempo e resultado
- Distingue exportar o modelo de renderizar um vídeo da animação

Tipos de item: `multipla_escolha` **Cursos de origem:** `UX03.01`, `UX03.05`

Cursos da trilha

UX03.01 — Blender: criando um modelo 3D

8h · 39 atividades · atualizado em 2026-07-08 | Etapa: *O Blender e o universo 3D*
https://www.alura.com.br/curso-online-blender-criando-primeiro-modelo-3d

Módulos da ementa. Conhecendo o Blender; Criando o ambiente; Texturizando; Renderizando.

Conceitos-chave. Blender, viewport, modo objeto, modo edição, blocagem, propriedades do objeto, modificador, iluminação, material, motor de render, exportação.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX03.01.O1	Reconhecer os painéis do Blender e navegar no espaço 3D do viewport	entender	UX03.C1
UX03.01.O2	Modelar um ambiente minimalista por blocagem a partir de primitivas	aplicar	UX03.C2
UX03.01.O3	Ajustar propriedades de objeto e aplicar modificadores à malha	aplicar	UX03.C4
UX03.01.O4	Iluminar a cena e escolher o motor de render adequado ao resultado desejado	aplicar	UX03.C8
UX03.01.O5	Renderizar e exportar a animação da composição 3D	aplicar	UX03.C8
UX03.01.O6	Reconhecer as limitações e as exigências de hardware do trabalho em 3D	entender	UX03.C1

Competências alimentadas por este curso: UX03.C1 Operar a interface do Blender e navegar no espaço 3D; UX03.C2 Modelar por blocagem e com Box Modeling; UX03.C4 Usar modificadores e propriedades de objeto; UX03.C8 Animar com keyframes e renderizar o resultado.

Fora de escopo (o questionário não cobra). escultura digital; simulação de fluidos e partículas; composição de vídeo; nodes de shader avançados.

UX03.02 — Blender: entendendo o Box Modeling

8h · 39 atividades · atualizado em 2026-07-08 | Etapa: *Modelagem 3D com Blender*
https://www.alura.com.br/curso-online-blender-entendendo-box-modeling

Módulos da ementa. Organizando o Workspace; Modelando a Lâmina; O cabo da espada; O altar.

Conceitos-chave. box modeling, concept, modelsheet, imagem de referência, extrusão, lowpoly, fluxo de malha, atalho de teclado, composição, render de portfólio.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX03.02.O1	Organizar a área de trabalho e inserir concept e modelsheet como referência	aplicar	UX03.C1
UX03.02.O2	Modelar um objeto lowpoly com Box Modeling partindo de uma primitiva	aplicar	UX03.C2
UX03.02.O3	Usar extrusão para desenvolver a forma a partir das faces selecionadas	aplicar	UX03.C2

Objetivo	Enunciado	Bloom	Competência
UX03.02.O4	Explicar o conceito de fluxo de malha e seu efeito sobre o modelo	entender	UX03.C3
UX03.02.O5	Usar atalhos de teclado para agilizar o trabalho de modelagem	aplicar	UX03.C1

Competências alimentadas por este curso: UX03.C1 Operar a interface do Blender e navegar no espaço 3D; UX03.C2 Modelar por blocagem e com Box Modeling; UX03.C3 Controlar o fluxo de malha e modelar Poly by Poly.

Fora de escopo (o questionário não cobra). modelagem hard surface avançada; CAD e modelagem para impressão 3D; escultura de alta densidade.

UX03.03 — Blender: texturizando modelos com Hand Paint

8h · 37 atividades · atualizado em 2026-07-08 | Etapa: *Modelagem 3D com Blender*
https://www.alura.com.br/curso-online-blender-texturizando-modelos-hand-paint

Módulos da ementa. Entendendo o UV Mapping; Organizando a UV da espada; Técnica de pintura Hand Painting; Criando o portfólio.

Conceitos-chave. UV mapping, UV editor, unwrap, costura, resolução de textura, material, Hand Painting, pintura digital, prop para game, portfólio 3D.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX03.03.O1	Explicar o que é o mapa UV e qual o seu papel na texturização	entender	UX03.C5
UX03.03.O2	Abrir a UV de um modelo com unwrap e testar a distribuição obtida	aplicar	UX03.C5
UX03.03.O3	Escolher a resolução de mapa adequada ao tamanho e ao uso do objeto	avaliar	UX03.C5
UX03.03.O4	Organizar os materiais e pintar a textura do modelo com a técnica Hand Paint	aplicar	UX03.C6
UX03.03.O5	Preparar o prop texturizado para ser levado a uma engine de jogo	aplicar	UX03.C6

Competências alimentadas por este curso: UX03.C5 Abrir e organizar o mapa UV; UX03.C6 Texturizar e pintar com a técnica Hand Paint.

Fora de escopo (o questionário não cobra). texturização procedural por nodes; baking de mapas normal e AO; PBR completo; Substance Painter.

UX03.04 — Blender: modelando com a técnica Poly by Poly

8h · 39 atividades · atualizado em 2026-07-08 | Etapa: *Modelagem 3D com Blender*
https://www.alura.com.br/curso-online-blender-modelando-tecnica-poly-by-poly

Módulos da ementa. Técnica poly by poly; Modelando a cabeça; Modelando o corpo; Modelando os acessórios; Textura e finalização.

Conceitos-chave. poly by poly, retopologia, fluxo de malha, separar e unir malha, modificador, modelagem de personagem, acessório, coloração.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX03.04.O1	Comparar as técnicas de modelagem 3D e identificar quando usar Poly by Poly	analisar	UX03.C3
UX03.04.O2	Modelar cabeça, corpo e membros de um personagem polígono a polígono	aplicar	UX03.C3
UX03.04.O3	Explicar o que é retopologia e em que situação ela é necessária	entender	UX03.C3
UX03.04.O4	Separar e unir partes diferentes de uma malha durante a modelagem	aplicar	UX03.C3
UX03.04.O5	Aplicar modificadores para facilitar o processo de modelagem do personagem	aplicar	UX03.C4
UX03.04.O6	Colorir o personagem finalizado e compor a cena com os acessórios	aplicar	UX03.C6

Competências alimentadas por este curso: UX03.C3 Controlar o fluxo de malha e modelar Poly by Poly; UX03.C4 Usar modificadores e propriedades de objeto; UX03.C6 Texturizar e pintar com a técnica Hand Paint.

Fora de escopo (o questionário não cobra). anatomia avançada; escultura em ZBrush; roupas com simulação de tecido; cabelo com sistema de partículas.

UX03.05 — Blender: riggando e animando seu personagem

8h · 38 atividades · atualizado em 2026-07-08 | Etapa: *Animação 3D com Blender*
https://www.alura.com.br/curso-online-blender-riggando-animando-personagem

Módulos da ementa. A animação no 3D; Animação por Bones; Pintura de peso; Finalizando a animação.

Conceitos-chave. keyframe, tipos de animação, bone, esqueleto, armature, IK, controlador, pintura de peso, vinculação de malha, NonLinear Animation, exportação de personagem.

Objetivos de aprendizagem

Objetivo	Enunciado	Bloom	Competência
UX03.05.O1	Distinguir os tipos de animação e explicar o papel dos keyframes	entender	UX03.C8
UX03.05.O2	Montar o esqueleto do personagem com bones alinhados à malha	aplicar	UX03.C7
UX03.05.O3	Criar IK e controladores para melhorar o fluxo de trabalho de animação	aplicar	UX03.C7
UX03.05.O4	Vincular a malha ao esqueleto e corrigir deformações pela pintura de peso	analisar	UX03.C7
UX03.05.O5	Animar o personagem e organizar os trechos com a NonLinear Animation	aplicar	UX03.C8

Objetivo	Enunciado	Bloom	Competência
UX03.05.O6	Exportar o personagem animado e gerar um vídeo da animação	aplicar	UX03.C8

Competências alimentadas por este curso: UX03.C7 Montar o rig do personagem; UX03.C8 Animar com keyframes e renderizar o resultado.

Fora de escopo (o questionário não cobra). motion capture; rigging facial; physics de corpo mole; animação procedural; retargeting de animação.

Rubrica de proficiência

Faixa	%	Descritor
iniciante	0–49	Reconhece a tela do Blender, mas confunde os modos de trabalho e não sabe por onde começar um modelo.
basico	50–69	Modela formas simples seguindo um passo a passo; não explica fluxo de malha nem para que serve o mapa UV.
adequado	70–84	Modela por blocagem e Box Modeling escolhendo a primitiva certa, abre e organiza o UV, pinta a textura à mão e monta o esqueleto de um personagem.
avancado	85–100	Justifica a técnica de modelagem pela peça e pelo uso, relaciona fluxo de malha à deformação na animação, diagnostica deformação errada pela pintura de peso e escolhe o motor de render com critério.

Nível mínimo esperado: adequado.

> **Observação da coordenação.** Faixas definidas pela coordenação ITT, não extraídas da Alura. Trilha de produção prática: o questionário mede o entendimento das decisões de modelagem, texturização e rigging, não a destreza no software — essa só se avalia pelo arquivo entregue. Revisar após o primeiro ciclo de aplicação.

Prompt do questionário

Blueprint. 20 itens, 40 minutos estimados. Por tipo na matriz: multipla_escolha 18 · dissertativa 2. Por nível cognitivo: lembrar/entender 7 · aplicar 9 · analisar/avaliar 4.

Roteiro de montagem. Percorra as linhas de cima para baixo: em cada uma, selecione o botão indicado na plataforma e peça a quantidade daquele tema.

Tema	Qtd.	Botão	Com estímulo	Foco da linha
C1 Interface e navegação 3D	2	Múltipla escolha	1	modo objeto vs modo edição, papel da referência
C2 Blocagem e Box Modeling	3	Múltipla escolha	1	primitiva de partida, o que a extrusão faz, para que serve a blocagem
C3 Fluxo de malha e Poly by Poly	2	Múltipla escolha	1	quando usar cada técnica, o que a retopologia resolve

Tema	Qtd.	Botão	Com estímulo	Foco da linha
C3 Fluxo de malha e Poly by Poly	1	Dissertativa	0	por que o fluxo de malha importa em um modelo que vai ser animado
C4 Modificadores e propriedades	1	Múltipla escolha	0	qual modificador produz o resultado, ordem na pilha
C4 Modificadores e propriedades	1	Múltipla resposta	0	quais afirmações sobre modificadores não destrutivos estão corretas
C5 Mapa UV	2	Múltipla escolha	1	o que o UV representa, textura esticada, resolução adequada
C6 Texturização Hand Paint	2	Múltipla escolha	1	material vs textura, emenda na costura do UV
C6 Texturização Hand Paint	1	Dissertativa	0	o que precisa estar pronto para levar o prop a uma engine
C7 Rig: bones, IK e pintura de peso	2	Múltipla escolha	1	deformação errada no braço, função da IK
C7 Rig: bones, IK e pintura de peso	1	Múltipla resposta	0	quais passos o rig precisa cumprir antes de animar
C8 Keyframes, render e exportação	2	Múltipla escolha	0	o que o keyframe marca, exportar o modelo vs renderizar vídeo

Totais: Múltipla escolha 16 · Múltipla resposta 2 · Dissertativa 2 = **20 itens**; 6 com estímulo no enunciado.

> Equivalência com a matriz: os 18 itens objetivos de ‘cursos/UX03-blender.json‘ (‘multipla_escolha‘ 18) aparecem aqui como 16 múltipla escolha + 2 múltipla resposta. A matriz não usa ‘analise_codigo‘ — a trilha não tem programação —, por isso a coluna aqui é "Com estímulo" e não "Com código".

Texto para colar na plataforma. Cole o bloco abaixo inteiro no campo de prompt, uma vez só. Ele define o *tema* das questões; o *tipo* vem do botão que você seleciona a cada linha do roteiro acima.

```
AVALIAÇÃO PÓS-TRILHA - BLENDER E MODELAGEM 3D

Contexto: questionário aplicado depois de a trilha ser concluída, em uma ONG que ensina
computação para adolescentes do ensino fundamental II e médio. A trilha é a primeira
experiência do estudante com modelagem 3D e vai da interface do Blender à animação de um
personagem. As peças construídas foram um ambiente minimalista blocado, uma espada lowpoly
modelada e pintada à mão, e um personagem modelado poly by poly, riggado e animado.

ASSUNTOS

C1 - Interface e navegação no espaço 3D
  Cobrar: dada uma tarefa, dizer em qual modo de trabalho ela se realiza (objeto ou edição);
  reconhecer o papel da imagem de referência e do modelsheet antes de começar a modelar;
  distinguir mover a câmera do viewport de mover o objeto na cena.
  Erros típicos, para virar alternativa errada: tentar editar vértices sem entrar no modo de
  edição; mover o objeto achando que está apenas girando a visualização; modelar sem
  referência e perder as proporções; confundir deletar o objeto com escondê-lo da
  visualização.

C2 - Blocagem e Box Modeling
```

```
  Cobrar: dada uma forma a construir, escolher a primitiva de partida adequada; explicar o
  que a extrusão faz com a face selecionada; reconhecer para que serve a blocagem antes do
  detalhamento.
  Erros típicos: começar pelo detalhe antes de a forma geral estar resolvida; escalar o
  objeto no modo objeto quando a alteração deveria ser na malha; extrudar sem perceber que
  criou faces internas sobrepostas; confundir lowpoly com modelo malfeito em vez de escolha
  de estilo.

C3 - Fluxo de malha e Poly by Poly
  Cobrar: dada uma peça a modelar, justificar a escolha entre Box Modeling e Poly by Poly;
  explicar por que o fluxo de malha importa em um modelo que vai ser animado; dizer em uma
  frase o que a retopologia resolve.
  Erros típicos: achar que o fluxo de malha é só questão de aparência, sem efeito na
  animação; deixar faces com mais de quatro lados onde a deformação vai acontecer; confundir
  retopologia com apagar e modelar de novo; separar partes da malha e esquecer de uni-las
  antes de continuar.

C4 - Modificadores e propriedades de objeto
  Cobrar: dado um resultado desejado, escolher o modificador que o produz; reconhecer que o
  modificador é não destrutivo até ser aplicado; explicar a vantagem de modelar metade e
  espelhar com modificador.
  Erros típicos: aplicar o modificador cedo e perder a possibilidade de ajustar; esperar que
  o modificador altere a malha base antes de ser aplicado; empilhar modificadores sem atentar
  para a ordem em que agem; modelar as duas metades à mão onde o espelhamento resolveria.

C5 - Mapa UV
  Cobrar: explicar em uma frase o que o mapa UV representa; reconhecer um UV mal distribuído
  pelo esticamento da textura no modelo; relacionar a resolução do mapa ao tamanho do objeto
  na cena.
  Erros típicos: pintar a textura antes de abrir o UV; marcar costuras em lugares visíveis
  do modelo; achar que o UV é gerado automaticamente sem ajuste; usar resolução altíssima em
  um prop pequeno e pesar o projeto sem ganho.

C6 - Texturização Hand Paint
  Cobrar: distinguir o material do modelo da textura aplicada a ele; reconhecer o que a
  pintura Hand Paint resolve que a cor lisa não resolve; saber o que precisa estar pronto
  para levar o prop a uma engine.
  Erros típicos: confundir atribuir uma cor ao material com pintar uma textura; pintar sem
  considerar onde estão as costuras do UV e ver a emenda aparecer; esquecer de exportar a
  textura junto com o modelo; achar que a iluminação da cena substitui o sombreamento
  pintado.

C7 - Rig: bones, IK, controladores e pintura de peso
  Cobrar: dada uma deformação errada no braço, apontar que a pintura de peso é a causa;
  explicar o que o controlador resolve no fluxo de trabalho de animação; reconhecer a função
  da IK em um membro do personagem.
  Erros típicos: animar os bones direto sem criar controladores; vincular a malha ao
  esqueleto e não revisar a pintura de peso; montar o esqueleto fora do eixo do personagem;
  confundir o bone com o controlador que o movimenta.

C8 - Keyframes, render e exportação
  Cobrar: explicar o que um keyframe marca e o que o Blender faz entre dois deles;
  reconhecer que a escolha do motor de render muda tempo e resultado; distinguir exportar o
  modelo de renderizar um vídeo da animação.
  Erros típicos: marcar keyframe sem perceber que ele registra a posição daquele quadro;
  esperar que a animação apareça no modelo exportado para uma engine sem configurá-la;
  renderizar em alta qualidade para testar e perder horas; esquecer a iluminação e renderizar
  uma cena escura.

FORA DE ESCOPO - não cobre nada disso: escultura digital, simulação de fluidos e partículas,
```

```
composição de vídeo, nodes de shader avançados, modelagem hard surface avançada, CAD e
modelagem para impressão 3D, texturização procedural por nodes, baking de mapas normal e AO,
PBR completo, Substance Painter, anatomia avançada, ZBrush, simulação de tecido, cabelo por
partículas, motion capture, rigging facial, animação procedural e retargeting.

REDAÇÃO E CALIBRAGEM

- Português brasileiro, linguagem direta, sem jargão corporativo. Enunciado de no máximo
  3 linhas.
- Contextos concretos e próximos do público: prop de jogo (espada, baú, poção), ambiente
  isométrico de um quarto, personagem de plataforma, veículo estilizado, cenário de floresta.
- Quando o item trouxer estímulo, descreva a situação de modelagem em texto - a forma a
  construir, o que apareceu de errado no modelo, a malha resultante - em no máximo 8 linhas.
  Nunca dependa de o estudante ver uma imagem ou um print da tela.
- Não cobre nome de botão, posição de painel nem atalho de teclado: avalie o entendimento da
  decisão de modelagem, texturização ou rigging, não a memória da interface.
- As alternativas erradas devem sair dos erros típicos listados em cada assunto, nunca ser
  aleatórias: toda alternativa errada precisa ser plausível para quem entendeu em parte.
- Não pergunte nada sobre a plataforma Alura, nomes de instrutores, duração de curso ou
  ordem dos módulos. Avalie raciocínio de produção 3D, não memória do curso.
- Calibragem cognitiva do conjunto: cerca de 7 itens de compreensão, 9 de aplicação e 4
  de análise ou avaliação.
```

Apêndice — fontes e reprodutibilidade

Os dados factuais de cada trilha — lista de cursos, carga horária, módulos da ementa, público-alvo e objetivos declarados — vêm das páginas públicas da Alura, sem login e sem baixar vídeo. Competências, pesos, erros comuns, faixas de proficiência e os roteiros de montagem são elaboração pedagógica da ITT e são, portanto, revisáveis.

Trilha	Extração	Fontes
PR01	2026-10-07	https://www.alura.com.br/formacao-programacao https://www.alura.com.br/curso-online-comecando-programacao-carreira-primeiros-passos https://www.alura.com.br/curso-online-logica-programacao-mergulhe-programacao-javascript https://www.alura.com.br/curso-online-logica-programacao-funcoes-listas https://www.alura.com.br/curso-online-git-github-compartilhando-colaborando-projetos https://www.alura.com.br/curso-online-logica-programacao-praticando-desafios imagens/planilha-trilhas-atual.jpeg
PR02	2026-10-07	https://www.alura.com.br/formacao-linguagem-python https://www.alura.com.br/curso-online-python-crie-sua-primeira-aplicacao https://www.alura.com.br/curso-online-python-aplicando-orientacao-objetos https://www.alura.com.br/curso-online-python-avance-orientacao-objetos-consuma-api imagens/planilha-trilhas-atual.jpeg
PR03	2026-10-21	https://www.alura.com.br/formacao-django https://www.alura.com.br/curso-online-django-templates-boas-praticas https://www.alura.com.br/curso-online-django-persistencia-dados-admin https://www.alura.com.br/curso-online-django-autenticacao-formularios-alerta https://www.alura.com.br/curso-online-django-autenticacao-oauth2-0 https://www.alura.com.br/curso-online-django-crud-persistencia-s3 imagens/planilha-trilhas-atual.jpeg
PR06	2026-10-21	https://www.alura.com.br/formacao-desenvolvendo-jogos-2d-unity imagens/planilha-trilhas-atual.jpeg
PR12	2026-10-21	https://www.alura.com.br/formacao-boas-praticas-python https://www.alura.com.br/curso-online-python-aplicando-boas-praticas-pep-8 https://www.alura.com.br/curso-online-python-construindo-sistemas-eficientes-solid-design-patterns https://www.alura.com.br/curso-online-python-tdd-explorando-testes-unitarios imagens/planilha-trilhas-atual.jpeg
FE01	2026-10-07	https://www.alura.com.br/formacao-html-css https://www.alura.com.br/curso-online-internet-entendendo-fundamentos-da-web https://www.alura.com.br/curso-online-html-css-ambiente-estrutura-estilo https://www.alura.com.br/curso-online-html-css-formularios-seo-acessibilidade https://www.alura.com.br/curso-online-html-css-responsividade-layouts imagens/planilha-trilhas-atual.jpeg

Trilha	Extração	Fontes
FE03	2026-10-21	https://www.alura.com.br/formacao-front-end https://www.alura.com.br/curso-online-construindo-paginas-dinamicas-javascript https://www.alura.com.br/curso-online-javascript-entendendo-promises-async-await https://www.alura.com.br/curso-online-javascript-metodos-array https://www.alura.com.br/curso-online-javascript-manipulando-elementos-dom https://www.alura.com.br/curso-online-javascript-manipulacao-elementos-localstorage https://www.alura.com.br/curso-online-javascript-implementando-crud-requisicoes-http https://www.alura.com.br/curso-online-javascript-evoluindo-sua-aplicacao-es6 https://www.alura.com.br/curso-online-node-js-terminal-ambiente-desenvolvimento-front-end imagens/planilha-trilhas-atual.jpeg
UX02	2026-10-21	https://www.alura.com.br/formacao-game-design https://www.alura.com.br/curso-online-game-design-definindo-principios-jogo-digital https://www.alura.com.br/curso-online-game-design-viabilizando-jogo-produto https://www.alura.com.br/curso-online-game-design-refinando-aspectos-visuais-artisticos-jogo https://www.alura.com.br/curso-online-game-design-estrategias-viabilizar-monetizar-seu-jogo imagens/planilha-trilhas-atual.jpeg
UX03	2026-10-21	https://www.alura.com.br/formacao-blender https://www.alura.com.br/curso-online-blender-criando-primeiro-modelo-3d https://www.alura.com.br/curso-online-blender-entendendo-box-modeling https://www.alura.com.br/curso-online-blender-texturizando-modelos-hand-paint https://www.alura.com.br/curso-online-blender-modelando-tecnica-poly-by-poly https://www.alura.com.br/curso-online-blender-riggando-animando-personagem imagens/planilha-trilhas-atual.jpeg

Ressalva de fonte — PR06. Carga horária, público e bullets 'O que você vai aprender' da página pública da formação na Alura. ATENÇÃO: as páginas individuais dos dois cursos (curso-online-unity-criando-jogo-metroidvania-2d e curso-online-unity-criando-menus-coletaveis-batalha-final-jogo-2d) não estão publicadas — redirecionam para a home da Alura em 2026-10-21. Por isso: a lista de módulos de cada curso foi derivada da descrição da etapa na página da formação, não de uma ementa publicada; a carga de 12h por curso é a divisão igual do total de 24h declarado para a formação; n_atividades e atualizado_em ficam nulos. Reconferir quando as páginas forem publicadas. Competências, pesos, erros comuns e faixas de proficiência são elaboração pedagógica da ITT.

A consistência de todos os arquivos é verificável com `python3 scripts/validar.py`, que confere, entre outras regras, se o roteiro de cada prompt fecha com o blueprint da sua matriz. Este relatório é gerado por `python3 scripts/gerar_relatorio.py` e não deve ser editado à mão.